

**PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE
INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN
MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS**

JOTA LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

**FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS
TULUÁ – VALLE DEL CAUCA**

2026

**PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE
INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN
MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS**

JOTA LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

Director

Carlos Andrés Delgado Saavedra, Ing.

carlos.andres.delgado@correounivalle.edu.co

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS

TULUÁ – VALLE DEL CAUCA

2026

Tabla de Contenido

Resumen	VI
Introducción	VII
1. Contexto y objetivos	1
1.1. Problema de Investigación	1
1.1.1. Descripción del problema	1
1.1.2. Formulación del problema	3
1.2. Objetivos	3
1.2.1. Objetivo general	3
1.2.2. Objetivos específicos y resultados esperados	3
1.3. Alcance	3
2. Marco Referencial	5
2.1. Glosario	5
2.2. Marco Teórico	6
2.2.1. Fundamentos de la Interpretación de Lenguajes de Programación	6
2.2.1.1. Gramáticas formales y notación BNF	6
2.2.1.2. Árbol de Sintaxis Abstracta	7
2.2.1.3. Proceso de interpretación y ambientes de ejecución	7
2.2.1.4. Racket y la librería EOPL como marco pedagógico	8
2.2.2. Teorías del Aprendizaje Aplicadas a la Enseñanza de Compiladores	9
2.2.2.1. Teoría de la carga cognitiva	9
2.2.2.2. Visualización de programas como apoyo al aprendizaje	9
2.2.2.3. Evaluación por pasos como estrategia pedagógica	10
2.2.3. Principios de Diseño de Herramientas Educativas Interactivas	10
2.2.3.1. Usabilidad en tecnología educativa	10
2.2.3.2. Organización visual y presentación simultánea de representaciones	10
2.2.4. Relación conceptual entre gramática, AST y ambiente de ejecución	11
2.3. Antecedentes	12
3. Diagnóstico de dificultades conceptuales	15
3.1. Metodología	15
3.1.1. Diseño de los instrumentos	15
3.1.2. Selección de la población	16

3.2. Análisis de la encuesta a estudiantes	16
3.2.1. Resultados cuantitativos	16
3.2.2. Resultados cualitativos	18
3.3. Análisis de la encuesta a docentes	18
3.3.1. Percepción general de los estudiantes	19
3.3.2. Dificultades con paradigmas avanzados	20
3.4. Revisión del microcurrículo	20
3.5. Selección de conceptos base para el prototipo	21
4. Diseño del prototipo FLP Viewer	23
4.1. Especificación de requisitos del sistema	23
4.1.1. Enfoque metodológico	23
4.1.2. Requisitos funcionales	24
4.1.3. Requisitos no funcionales	24
4.2. Diseño del prototipo	25
4.2.1. Arquitectura del sistema	25
4.2.1.1. Infraestructura de despliegue	26
4.2.1.2. Capa de servidor	26
4.2.1.3. Capa de cliente	26
4.2.1.4. Pipeline de gramática	27
4.2.2. Selección de tecnologías	27
4.2.3. Prototipos de interfaz de usuario	28
4.2.4. Estrategias de visualización	31
5. Implementación del prototipo FLP Viewer	33
5.1. Repositorio de código fuente	33
5.2. Implementación de los componentes	34
5.2.1. Pipeline de gramática BNF	34
5.2.2. Componente orquestador y paneles de la interfaz	35
5.2.3. Biblioteca de ejemplos predefinidos	39
5.2.4. Punto de acceso de ejecución	41
5.2.5. Instrumentación en tiempo de ejecución	42
5.3. Flujo de ejecución principal	42
5.4. Despliegue en entorno de pruebas	45
5.5. Verificación de requisitos funcionales	46
Bibliografía	47
A. Microcurrículo de FLP	50
B. Encuesta a estudiantes	51
C. Encuesta a docentes	52
D. Protocolo de búsqueda sistemática de literatura	53
E. Requisitos del sistema	54

TABLA DE CONTENIDO

III

E.1. Requisitos Funcionales	54
E.2. Requisitos No Funcionales	58

Lista de Figuras

1.1. Resultados de la encuesta aplicada a estudiantes de FLP (n=36)	2
2.1. Relación conceptual entre los componentes del proceso de interpretación en el modelo EOPL. La fila superior muestra el pipeline de análisis (tiempo de compilación); la fila inferior, el pipeline de ejecución (tiempo de evaluación). La flecha discontinua indica que la gramática determina los constructores del AST en tiempo de diseño.	11
4.1. Diagrama de arquitectura del sistema	27
4.2. Wireframe de la interfaz principal del prototipo FLP Viewer	30
4.3. Wireframe del modal de definición de gramática BNF	31
5.1. Modal de bienvenida del prototipo FLP Viewer	35
5.2. Modo de evaluación paso a paso con pasos pendientes de navegación	36
5.3. Modal de definición de gramática BNF con vista previa del archivo generado	37
5.4. Evaluación de una expresión con iteración y estado mutable: suma acumulada mediante bucle <code>for</code>	38
5.5. Evaluación de una función recursiva con clausura: la ligadura <code>f</code> en el panel de ambiente refleja una función como valor de primera clase	39
5.6. Diagrama de secuencia del flujo principal de ejecución del prototipo FLP Viewer	44
5.7. Diagrama de despliegue del prototipo FLP Viewer en el entorno de pruebas	46

Lista de tablas

1.1. Objetivos Específicos	3
2.1. Síntesis comparativa de herramientas relacionadas con la enseñanza de compiladores e intérpretes (2021–2026)	12
3.1. Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$)	17
3.2. Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$)	19
3.3. Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$)	20
3.4. Indicadores de logro seleccionados como base para el prototipo	22
4.1. Resumen de requisitos funcionales del sistema	24
4.2. Resumen de requisitos no funcionales del sistema	25
4.3. Tecnologías seleccionadas y justificación	28
5.1. Biblioteca de ejemplos predefinidos y correspondencia con indicadores de logro	39
5.2. Verificación del cumplimiento de los requisitos funcionales	46
E.1. RF-01: Generación del AST	54
E.2. RF-02: Visualización gráfica e interactiva del AST	55
E.3. RF-03: Ejecución y evaluación del programa	55
E.4. RF-04: Representación del ambiente de ejecución	56
E.5. RF-05: Biblioteca de ejemplos	56
E.6. RF-06: Soporte y validación de sintaxis en Racket	57
E.7. RF-07: Visualización de los componentes internos del intérprete	57
E.8. RF-08: Plantilla base editable y exportación del entorno	58
E.9. RNF-01: Usabilidad	59
E.10. RNF-02: Rendimiento	59
E.11. RNF-03: Accesibilidad web	60
E.12. RNF-04: Mantenibilidad	60
E.13. RNF-05: Escalabilidad	61

Resumen

// TODO: Escribir el resumen del proyecto, incluyendo los objetivos, metodología y resultados esperados.

Introducción

// TODO: Introducir la problemática y el contexto del estudio.

Capítulo 1

Contexto y objetivos

1.1. Problema de Investigación

1.1.1. Descripción del problema

La interpretación y compilación de lenguajes de programación constituye uno de los ejes centrales en la formación de ingenieros de sistemas. Comprender cómo un programa escrito en un lenguaje de alto nivel es procesado, analizado y ejecutado por un sistema computacional permite sustentar el trabajo en múltiples áreas de la disciplina, desde el desarrollo de compiladores y herramientas de análisis estático, hasta el diseño de lenguajes de dominio específico. Sin embargo, la enseñanza de estos contenidos enfrenta dificultades por sus conceptos abstractos, densos y difíciles de conectar con la práctica [1] [2].

En el programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá, la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) demanda una comprensión teórica sólida y una capacidad de visualización de procesos internos que no resultan evidentes a partir del código fuente [3]. A lo largo de su desarrollo académico, los estudiantes deben asimilar conceptos como el análisis léxico y sintáctico, la construcción y recorrido de árboles de sintaxis abstracta (AST), la representación de ambientes de ejecución, la gestión del alcance léxico y el modelado de clausuras.

La principal herramienta de trabajo de la asignatura es el lenguaje Racket junto con los materiales del libro *Essentials of Programming Languages* (EOPL) [4]. Si bien esta combinación ofrece una base conceptual rigurosa y permite implementar intérpretes funcionales, su aprovechamiento exige una familiaridad previa con el entorno, así como un dominio sólido del paradigma funcional, en el que los estudiantes pueden presentar ciertos vacíos conceptuales. En particular, la sintaxis de prefijo de Racket, su estructura basada en paréntesis y los identificadores propios del entorno tienden a desplazar la atención cognitiva del estudiante desde los conceptos fundamentales de interpretación hacia la resolución de errores sintácticos del lenguaje utilizado [5].

Además de la carga impuesta por el entorno de programación, la asignatura carece de recursos interactivos que permitan a los estudiantes visualizar de manera dinámica y progresiva los procesos internos que se estudian. La literatura en tecnología educativa ha demostrado consistentemente que las representaciones visuales e interactivas favorecen la comprensión de procesos de compilación e

interpretación al hacer tangibles las transformaciones que ocurren en cada etapa del procesamiento de un programa [6] [7]. En ausencia de estos recursos, el aprendizaje depende en gran medida de las explicaciones del docente en clase, lo que reduce la autonomía del estudiante y limita las posibilidades de exploración fuera del aula.

Con el propósito de caracterizar estas dificultades desde la experiencia directa de quienes han cursado la asignatura, se aplicaron dos encuestas: una dirigida a 36 estudiantes que asistieron en distintos períodos académicos, y otra a cinco docentes del programa. Los resultados de ambos instrumentos se analizan en detalle en el Capítulo 3; no obstante, los datos de la encuesta estudiantil permiten fundamentar el planteamiento del problema con evidencia cuantitativa directa.

Como se aprecia en la Figura 1.1, tres hallazgos resultan especialmente significativos. Primero, el 77.8% de los estudiantes reportó haber tenido dificultades para comprender conceptos abstractos como entornos, estado o secuenciación, lo que confirma el carácter problemático de los contenidos de mayor abstracción. Segundo, apenas el 19.4% se siente preparado para aplicar los conceptos aprendidos en otros cursos o proyectos, lo que revela una brecha importante entre la comprensión teórica y la transferencia práctica del conocimiento. Tercero, el 94.5% de los encuestados consideró que recursos adicionales, como laboratorios interactivos, herramientas de visualización o ejemplos prácticos progresivos, habrían mejorado su aprendizaje de manera significativa.

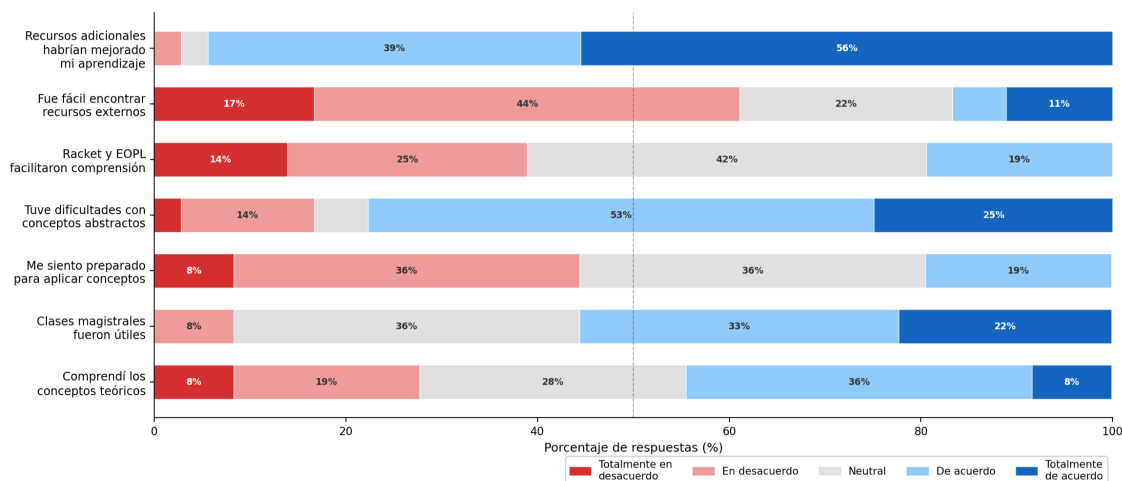


Figura 1.1: Resultados de la encuesta aplicada a estudiantes de FLP (n=36)

Fuente: elaboración propia

En síntesis, las dificultades observadas en la asignatura FLP responden a una unión de factores: el alto nivel de abstracción inherente a los contenidos, la fricción cognitiva que genera el entorno de programación utilizado, la escasez de recursos externos de apoyo, y la ausencia de herramientas interactivas que permitan a los estudiantes explorar y visualizar los procesos que estudian. Esta situación genera dependencia de las clases presenciales, limita la autonomía en el aprendizaje y se traduce en una comprensión superficial de conceptos fundamentales para asignaturas posteriores y para el ejercicio profesional.

1.1.2. Formulación del problema

¿Cómo desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis?

1.2. Objetivos

1.2.1. Objetivo general

Desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis.

1.2.2. Objetivos específicos y resultados esperados

Objetivo específico	Sección del documento
Identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura, con el propósito de orientar la selección de ejemplos, casos de uso y mecanismos de visualización.	Capítulo -
Caracterizar los fundamentos teóricos y técnicos relacionados con las gramáticas formales, el análisis sintáctico y la generación de árboles de sintaxis abstracta, que servirán como base conceptual para el diseño del prototipo.	Capítulo -
Diseñar la arquitectura web y las interfaces de usuario, integrando estrategias de visualización que favorezcan la comprensión de las estructuras sintácticas.	Capítulo -
Implementar un prototipo funcional que permita definir gramáticas, visualizar los árboles de sintaxis abstracta y los cambios en el estado durante la evaluación de los programas escritos por el usuario.	Capítulo -
Evaluar la usabilidad y funcionalidad del prototipo mediante pruebas con estudiantes inscritos en la asignatura y con el docente responsable.	Capítulo -

Tabla 1.1: Objetivos Específicos

1.3. Alcance

El presente trabajo está enfocado en el diseño e implementación de un prototipo web educativo orientado a apoyar la comprensión de los conceptos fundamentales de interpretación y compilación de lenguajes de programación en la asignatura FLP del programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá.

Desde el punto de vista técnico, el prototipo cubre tres funcionalidades principales: (1) la definición

y edición de gramáticas con sintaxis configurable por el usuario; (2) la visualización de árboles de sintaxis abstracta (AST) generados a partir de dichas gramáticas; y (3) la representación de los cambios en el ambiente durante la evaluación de programas escritos bajo la gramática definida. Estas funcionalidades están orientadas específicamente a los conceptos que, conforme al diagnóstico presentado en el Capítulo 3, presentan mayor dificultad de comprensión en la asignatura.

El alcance del prototipo se limita a funcionalidades demostrativas con fines pedagógicos. No se pretende desarrollar un lenguaje de propósito general ni implementar mecanismos avanzados de ejecución tales como depuración paso a paso, compilación a código máquina, optimización de rendimiento o autenticación y persistencia de usuarios. El sistema tampoco contempla despliegue en infraestructura institucional de uso permanente, puesto que su operación se restringe a un entorno de pruebas durante la fase de evaluación.

La evaluación del prototipo se llevará a cabo con un grupo de entre 10 y 20 estudiantes matriculados en la asignatura durante el período de desarrollo del trabajo, junto con el docente responsable. Dada la naturaleza y el tiempo disponible del proyecto, la evaluación se restringe a aspectos de funcionalidad técnica y usabilidad, mediante la aplicación del cuestionario estandarizado System Usability Scale (SUS) y un cuestionario de utilidad pedagógica de elaboración propia, dejando por fuera evaluaciones de impacto en el aprendizaje a largo plazo, pues estos requieren condiciones experimentales que exceden el alcance de un trabajo de grado.

Finalmente, el prototipo no pretende reemplazar los recursos actuales de la asignatura, en particular lo referente a Racket y EOPL, sino complementarlos, ofreciendo un punto de entrada más accesible y visual a los conceptos que los estudiantes identifican como los de mayor dificultad.

Capítulo 2

Marco Referencial

2.1. Glosario

Los términos que se definen a continuación son empleados de forma recurrente a lo largo del documento. Su inclusión tiene por objeto establecer un vocabulario preciso y evitar ambigüedades en la lectura de los capítulos posteriores.

- **Árbol de sintaxis abstracta (AST):** Representación jerárquica de la estructura semántica de un programa, producida por el analizador sintáctico a partir del código fuente. Elimina los detalles superficiales de la notación concreta y conserva únicamente la información relevante para la evaluación del programa. En el modelo de EOPL, los nodos del AST corresponden a los constructores del tipo de datos definido mediante `define-datatype` [4].
- **Ambiente de ejecución (*environment*):** Estructura de datos que asocia identificadores con sus valores durante la evaluación de un programa. Se organiza como una cadena de marcos (*frames*), donde cada llamada a `extend-env` agrega un nuevo marco con las ligaduras introducidas por la expresión en evaluación [4].
- **BNF (notación de Backus-Naur):** Metalenguaje estándar para la especificación de gramáticas libres de contexto mediante reglas de producción de la forma $\langle \text{no-terminal} \rangle ::= \alpha_1 \mid \alpha_2 \mid \dots$. En el prototipo propuesto en este trabajo, el estudiante utiliza notación BNF para definir la gramática de su intérprete [8].
- **Clausura (*closure*):** Valor que resulta de evaluar una expresión de función y que encapsula el cuerpo de la función junto con el ambiente léxico en el que fue definida. Las clausuras son la representación concreta de los procedimientos como valores de primera clase en el modelo de EOPL [4].
- **Carga cognitiva:** Cantidad de esfuerzo mental requerido para procesar información en la memoria de trabajo. La teoría distingue entre carga intrínseca (complejidad inherente del contenido), extrínseca (generada por la forma de presentación) y germana (asociada a la formación de esquemas de conocimiento) [9].
- **EOPL (*Essentials of Programming Languages*):** Libro de texto de Friedman y Wand que presenta una metodología para la construcción de intérpretes y compiladores en Racket

usando la librería SLLGEN [4]. Es el texto base de la asignatura FLP.

- **FLP:** Abreviatura empleada en este documento para referirse a la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (código 750017C) del programa de Ingeniería de Sistemas de la Universidad del Valle.
- **Indicador de logro (IL):** Criterio de evaluación del microcurrículo de la asignatura FLP que especifica el desempeño observable esperado en relación con un resultado de aprendizaje particular.
- **Ligadura (*binding*):** Asociación entre un identificador y un valor dentro de un marco del ambiente de ejecución. En el modelo funcional de EOPL, las ligaduras son inmutables: una vez creadas, no pueden modificarse; únicamente pueden ocultarse mediante la creación de un nuevo marco con el mismo identificador en el ambiente extendido [4].
- **Referencia implícita (*implicit reference*):** Modelo de ambiente introducido en el Capítulo 4 de EOPL en el que los valores se almacenan en celdas mutables representadas mediante referencias, y el ambiente contiene punteros a dichas celdas. Este modelo habilita la asignación destructiva mediante **set** sin crear un nuevo marco en el ambiente [4].
- **SLLGEN:** Biblioteca de análisis sintáctico incluida en Racket que genera automáticamente un escáner y un analizador a partir de una especificación léxica y una gramática BNF. Es la herramienta utilizada en EOPL para automatizar las etapas de análisis léxico y sintáctico en la construcción de intérpretes [4].

2.2. Marco Teórico

El presente proyecto se apoya en tres áreas de conocimiento: los fundamentos técnicos de la interpretación de lenguajes de programación, que constituyen el objeto de estudio que el prototipo busca hacer visible, las teorías del aprendizaje aplicadas a la educación en computación, que orientan las decisiones pedagógicas del diseño, y los principios de usabilidad y visualización en herramientas educativas interactivas, que guían las decisiones de interfaz. Los apartados siguientes desarrollan cada una de estas áreas.

2.2.1. Fundamentos de la Interpretación de Lenguajes de Programación

2.2.1.1. Gramáticas formales y notación BNF

Los lenguajes de programación se definen formalmente mediante gramáticas libres de contexto, clasificadas por Chomsky en el nivel dos de su jerarquía de lenguajes formales [10]. Una gramática libre de contexto se compone de un conjunto de símbolos terminales, un conjunto de símbolos no terminales, un símbolo inicial y un conjunto de reglas de producción que especifican cómo cada símbolo no terminal puede reemplazarse por una secuencia de símbolos terminales y no terminales. Esta estructura proporciona una descripción precisa y no ambigua de la sintaxis del lenguaje, que puede ser procesada de forma automática por un analizador sintáctico.

La notación de Backus-Naur (BNF) es un metalenguaje estándar para expresar gramáticas libres de contexto de forma legible para el ser humano [8]. En BNF, cada regla de producción adopta la forma $\langle \text{no-terminal} \rangle ::= \alpha_1 \mid \alpha_2 \mid \dots$, donde las alternativas están separadas por el símbolo $\mid$. Esta

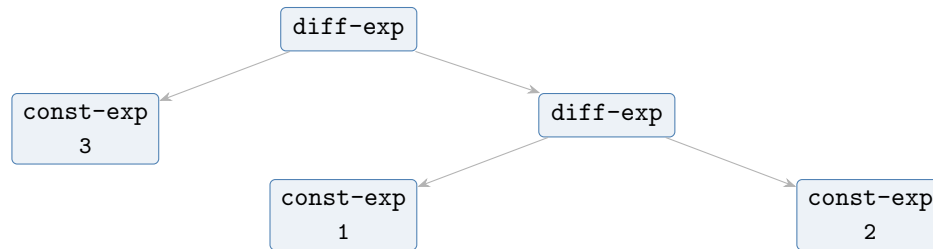
notación es el punto de entrada con el que el estudiante define la gramática de su intérprete en el prototipo propuesto: a partir de las reglas escritas en el área de definición de gramática, el pipeline genera automáticamente los archivos de código Racket necesarios para que SLLGEN construya el analizador correspondiente.

En el contexto de la librería EOPL, la gramática BNF se utiliza directamente como especificación de entrada para SLLGEN, la biblioteca de análisis sintáctico incluida en Racket [4]. A partir de dicha especificación, SLLGEN genera de forma automática un escáner y un analizador sintáctico, de modo que el estudiante puede concentrarse en la implementación de la semántica de su lenguaje sin necesidad de desarrollar manualmente las etapas de análisis léxico y sintáctico.

2.2.1.2. Árbol de Sintaxis Abstracta

El análisis sintáctico transforma la secuencia de tokens producida por el escáner en un árbol de sintaxis abstracta (AST), que representa la estructura jerárquica y semántica del programa con independencia de los detalles superficiales de la notación concreta [8]. A diferencia del árbol de derivación, el AST elimina elementos como paréntesis, palabras clave auxiliares y otros tokens cuya función es únicamente desambiguar la notación, pero que no contribuyen al significado del programa.

En el modelo de EOPL, el AST se representa mediante un tipo de datos algebraico definido con la macro `define-datatype` de Racket [4]. Cada regla de producción de la gramática se corresponde con un constructor del tipo de datos, de modo que existe una relación directa y verificable entre la estructura de la gramática BNF y la estructura del árbol. Esta correspondencia es pedagógicamente relevante porque permite al estudiante observar en el visualizador de AST del prototipo cómo cada construcción del programa origina un nodo específico del árbol, haciendo visible la relación entre la especificación formal del lenguaje y la representación interna que el intérprete utiliza para evaluar expresiones.



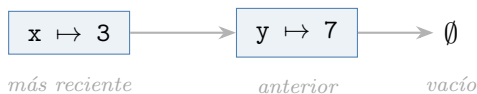
AST de (diff 3 (diff 1 2)): cada producción BNF origina un constructor del tipo de datos.

2.2.1.3. Proceso de interpretación y ambientes de ejecución

La interpretación de un programa consiste en recorrer el AST de forma recursiva y calcular el valor de cada nodo de acuerdo con las reglas semánticas del lenguaje [4]. En el modelo de EOPL, esta operación está encapsulada en la función `eval-expression`, que despacha sobre el tipo de cada nodo del árbol mediante la forma `cases`. Para cada constructor del tipo de datos, existe un caso que especifica cómo calcular el valor correspondiente, posiblemente invocando de forma recursiva a `eval-expression` sobre los subárboles del nodo.

Durante la evaluación, el intérprete mantiene un ambiente de ejecución que asocia identificadores con sus valores. Friedman y Wand definen el ambiente como una función parcial de identificadores

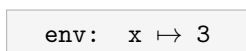
a valores denotados, y la operación fundamental sobre él es la búsqueda mediante **apply-env**, que recorre la cadena de marcos desde el más reciente hasta el ambiente inicial [4]. La extensión del ambiente mediante **extend-env** agrega un nuevo marco con las ligaduras introducidas por la expresión en evaluación, sin modificar los marcos preexistentes en el modelo funcional.



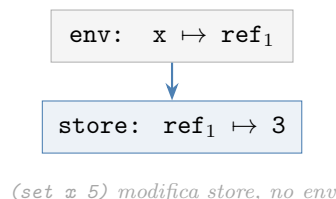
*Cadena de marcos tras (**extend-env** 'x 3 (**extend-env** 'y 7 (**empty-env**))). **apply-env** recorre de izquierda a derecha.*

A partir del Capítulo 4 de EOPL, el modelo introduce las referencias implícitas (IMPLICIT-REFS): los valores dejan de almacenarse directamente en el ambiente y pasan a residir en celdas mutables representadas por referencias. Esta modificación habilita la asignación destructiva mediante **set**, que modifica el contenido de una celda sin agregar un nuevo marco al ambiente. La diferencia entre ligadura y asignación que emerge de este cambio arquitectónico es uno de los conceptos de mayor dificultad en cursos que siguen el modelo de EOPL, dado que requiere distinguir entre dos operaciones que en la superficie parecen equivalentes pero tienen efectos estructuralmente distintos sobre el ambiente [11].

sin implicit-refs



con implicit-refs



2.2.1.4. Racket y la librería EOPL como marco pedagógico

Racket es un lenguaje de programación diseñado con propósitos docentes y de investigación en lenguajes de programación [12]. Su característica distintiva es la capacidad de modificar y extender el propio lenguaje mediante macros, lo que lo convierte en una plataforma especialmente adecuada para la implementación de intérpretes. En el contexto de la asignatura FLP, Racket se utiliza como plataforma para la construcción de los intérpretes descritos en EOPL, aprovechando su sistema de módulos y las herramientas de la librería para la generación automática de analizadores.

La librería EOPL proporciona, además de SLLGEN, las macros **define-datatype** y **cases**, que permiten definir tipos de datos algebraicos y despachar sobre ellos de forma idiomática [4]. Al declarar explícitamente los constructores del AST como constructores de un tipo de datos algebraico, el estudiante mantiene una correspondencia directa entre la gramática del lenguaje y la estructura del evaluador. En conjunto, estas herramientas permiten que la atención del estudiante se concentre en la semántica del lenguaje más que en los aspectos mecánicos del análisis sintáctico o la gestión de estructuras de datos, lo cual es coherente con el principio de reducción de la carga cognitiva extrínseca descrito en la sección siguiente [9].

2.2.2. Teorías del Aprendizaje Aplicadas a la Enseñanza de Compiladores

El diseño pedagógico del prototipo se fundamenta en tres corrientes teóricas que la literatura especializada ha identificado como relevantes para la enseñanza de contenidos de alta abstracción en ciencias de la computación: la teoría de la carga cognitiva, la visualización de programas como estrategia de apoyo al aprendizaje, y la evaluación por pasos como mecanismo para hacer observable el proceso de interpretación.

2.2.2.1. Teoría de la carga cognitiva

La teoría de la carga cognitiva, desarrollada por Sweller, establece que la memoria de trabajo posee una capacidad limitada para procesar información nueva de forma simultánea [9]. La teoría distingue tres tipos de carga: la intrínseca, determinada por la complejidad inherente del contenido y la interacción entre sus elementos; la extrínseca, generada por la forma en que la información se presenta; y la germana, asociada a los procesos mentales que contribuyen a la formación de esquemas de conocimiento duraderos. El diseño instruccional efectivo busca mantener la carga intrínseca en un nivel manejable, minimizar la carga extrínseca y maximizar la germana.

En el contexto de la enseñanza de compiladores e intérpretes, la carga intrínseca es elevada porque los conceptos involucrados, como gramáticas, AST, ambientes y clausuras, son abstractos y están interrelacionados de forma no trivial [11]. Esta complejidad hace especialmente necesario controlar la carga extrínseca mediante decisiones de diseño que faciliten la interpretación de la información presentada al estudiante. Cuando el estudiante debe coordinar fuentes dispersas de información durante su trabajo, como una especificación de gramática en un archivo, el código del evaluador en otro y la salida del intérprete en la consola del sistema operativo, la presión sobre la memoria de trabajo aumenta y los recursos cognitivos disponibles para la comprensión del contenido disminuyen [9]. El diseño de herramientas de apoyo para este tipo de cursos debe buscar reducir esa carga extrínseca integrando en un único entorno las representaciones relevantes del proceso de interpretación.

2.2.2.2. Visualización de programas como apoyo al aprendizaje

La visualización de programas es un área de investigación que estudia el uso de representaciones gráficas para apoyar la comprensión de conceptos abstractos de programación. Sorva et al. realizaron una revisión sistemática de los sistemas de visualización de programas para la enseñanza introductoria y concluyeron que la efectividad de estas herramientas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [13]. Cuando la visualización refleja con fidelidad las estructuras que el docente describe en clase, el estudiante puede establecer correspondencias directas entre la teoría y la herramienta, favoreciendo la comprensión y reduciendo el riesgo de formarse concepciones erróneas.

Python Tutor, desarrollado por Guo [14], es uno de los sistemas de visualización de ejecución más utilizados en educación en computación. La herramienta representa paso a paso el estado del ambiente de ejecución, incluyendo los marcos de activación, el montículo de objetos y los punteros entre ellos, para lenguajes como Python, Java y JavaScript. Su diseño demostró que la representación explícita del ambiente de ejecución facilita la comprensión de conceptos como el alcance de variables, la recursión y las referencias entre objetos. Sin embargo, Python Tutor no admite Racket ni visualiza la estructura interna de los intérpretes definidos por el usuario, lo que lo hace insuficiente para las necesidades específicas de la asignatura FLP.

La herramienta presentada por Spigariol et al. [5] constituye el antecedente más cercano al prototipo desarrollado en este trabajo en cuanto a objetivo pedagógico: combina un intérprete de un lenguaje funcional con componentes gráficos que visualizan las estructuras intermedias del proceso de interpretación. No obstante, dicha herramienta está diseñada para un lenguaje predefinido y no permite al estudiante definir su propia gramática ni trabajar con el modelo de EOPL y SLLGEN.

2.2.2.3. Evaluación por pasos como estrategia pedagógica

Clements, Flatt y Felleisen propusieron el modelo del *stepper* algebraico como herramienta para hacer observable el proceso de reducción de expresiones en lenguajes funcionales [15]. En este modelo, la evaluación de una expresión se presenta como una secuencia de pasos de reducción, donde cada paso muestra el estado del programa antes y después de la aplicación de una regla semántica. Este mecanismo permite al estudiante observar la correspondencia entre las reglas del evaluador y el comportamiento concreto del intérprete expresión por expresión.

La presentación de la evaluación como una secuencia de pasos discretos tiene además fundamento en la teoría de la carga cognitiva: al exponer un único paso de reducción a la vez, la herramienta controla la cantidad de información nueva que el estudiante debe procesar de forma simultánea, reduciendo la presión sobre la memoria de trabajo y favoreciendo la comprensión progresiva del comportamiento del intérprete [9]. El prototipo desarrollado en este trabajo implementa este principio mediante el modo de evaluación paso a paso descrito en el Capítulo 4.

2.2.3. Principios de Diseño de Herramientas Educativas Interactivas

2.2.3.1. Usabilidad en tecnología educativa

La usabilidad se define como el grado en que un sistema puede ser utilizado por usuarios específicos para alcanzar objetivos concretos con efectividad, eficiencia y satisfacción en un contexto de uso determinado. En el ámbito de las herramientas educativas digitales, investigaciones recientes señalan que la usabilidad no incide únicamente en la eficiencia técnica sino también en la motivación y la retención del aprendizaje: una interfaz que genera fricción desplaza los recursos cognitivos del usuario desde la comprensión del contenido hacia la resolución de problemas de navegación o interacción [7]. Esta relación es especialmente relevante en herramientas para la enseñanza de compiladores, donde la carga cognitiva intrínseca ya es elevada.

En el ámbito de los cursos de compiladores, investigaciones previas han documentado que los estudiantes perciben las herramientas de desarrollo convencionales como una fuente adicional de dificultad, independientemente de la solidez técnica de los contenidos [11]. Cuando la interfaz de una herramienta genera fricción, los recursos cognitivos se desvían desde la comprensión del contenido hacia la resolución de problemas de interacción, lo que reduce la capacidad disponible para el aprendizaje [7]. Por este motivo, el diseño de herramientas educativas para compiladores debe priorizar la minimización de los cambios de contexto y la integración de retroalimentación inmediata sobre los errores del estudiante.

2.2.3.2. Organización visual y presentación simultánea de representaciones

Norman señala que el diseño de interfaces debe respetar los límites de la atención humana, presentando la información de forma que la percepción y la interpretación requieran el mínimo esfuerzo

posible [16]. En el diseño de herramientas educativas para compiladores, este principio se traduce en la presentación simultánea de representaciones complementarias del proceso de interpretación, de modo que el estudiante pueda relacionar el código fuente con el AST y el ambiente sin necesidad de cambiar de ventana ni de contexto mental.

La literatura sobre herramientas educativas para compiladores confirma que la presentación simultánea de representaciones sincronizadas favorece la comprensión [3, 6]. Además, la diferenciación visual mediante color y estructura tipográfica permite al estudiante identificar la naturaleza de cada elemento de la interfaz sin necesidad de leer su contenido completo, lo que reduce el esfuerzo de interpretación y contribuye a mantener los recursos cognitivos disponibles para el análisis del comportamiento del intérprete [17].

2.2.4. Relación conceptual entre gramática, AST y ambiente de ejecución

Los apartados anteriores describen por separado las estructuras y operaciones que conforman el proceso de interpretación en el modelo EOPL: la gramática BNF como especificación formal del lenguaje, el AST como representación interna que el analizador produce a partir de esa especificación, y el ambiente de ejecución como estructura dinámica que el evaluador consulta y extiende al recorrer el árbol. La figura 2.1 sintetiza estas relaciones y explicita la cadena que constituye el núcleo conceptual del curso: desde que el estudiante escribe una gramática hasta que el intérprete produce un valor y deja un rastro en el ambiente.

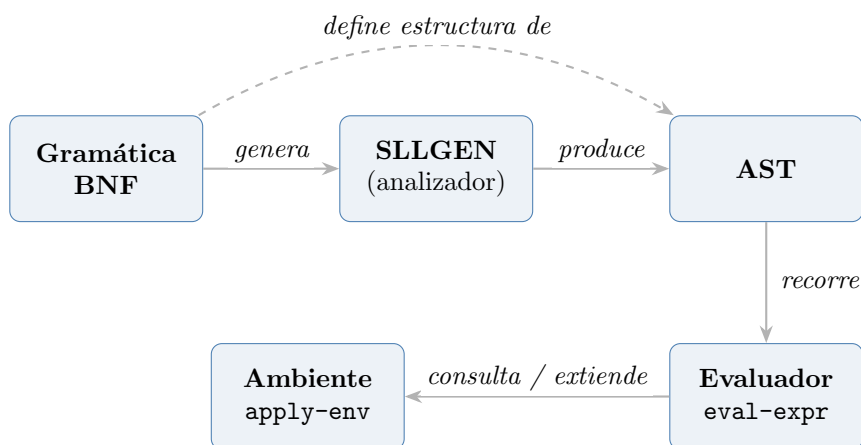


Figura 2.1: Relación conceptual entre los componentes del proceso de interpretación en el modelo EOPL. La fila superior muestra el pipeline de análisis (tiempo de compilación); la fila inferior, el pipeline de ejecución (tiempo de evaluación). La flecha discontinua indica que la gramática determina los constructores del AST en tiempo de diseño.

La flecha discontinua entre la gramática y el AST refleja la correspondencia estática descrita en la sección 2.2.1.2: cada producción BNF se convierte en un constructor del tipo de datos algebraico, de modo que la estructura del árbol es un reflejo directo de las decisiones que el estudiante toma al definir la gramática. Las flechas sólidas, en cambio, representan el flujo dinámico de la interpretación: SLLGEN transforma el código fuente en un AST, el evaluador lo recorre de forma recursiva y, a medida que lo hace, consulta y extiende el ambiente. Esta distinción entre estructura estática y comportamiento dinámico es una de las fuentes de dificultad identificadas en la literatura para

cursos de lenguajes de programación [11], y es precisamente lo que el prototipo propuesto busca hacer visible al presentar de forma simultánea el AST y el estado del ambiente tras cada paso de evaluación.

2.3. Antecedentes

La identificación de trabajos previos se realizó mediante una búsqueda sistemática en IEEE Xplore, ACM Digital Library y Scopus, acotada al período 2021–2026, utilizando términos relacionados con visualización de intérpretes, ambientes de ejecución, enseñanza de compiladores y herramientas educativas para cursos de lenguajes de programación. Los criterios de inclusión exigieron que los trabajos presentaran una herramienta o prototipo con componente de visualización o interactividad orientado a la enseñanza de alguna fase del proceso de interpretación o compilación, y que reportaran al menos una experiencia de uso o evaluación. El protocolo completo de búsqueda, incluyendo las cadenas booleanas utilizadas y el conteo de resultados por base, se presenta en el Anexo D.

Las propuestas encontradas pueden clasificarse en dos grupos: las que visualizan el pipeline de compilación clásico (análisis léxico, autómatas, tablas de análisis sintáctico) y las que apoyan específicamente la visualización del ambiente de ejecución y la evaluación de expresiones en lenguajes funcionales. La Tabla 2.1 presenta una síntesis comparativa de las herramientas más relevantes identificadas.

Tabla 2.1: Síntesis comparativa de herramientas relacionadas con la enseñanza de compiladores e intérpretes (2021–2026)

Autor / Año	Herramienta	Aporte principal	Limitación / Vacío
Jovanović et al. (2021) [6]	ComVIS	Entorno web interactivo para visualizar análisis léxico, construcción de autómatas y análisis sintáctico LL/LR con retroalimentación paso a paso. Reporta mejoras en comprensión y desempeño académico medidas mediante cuestionarios pre/post.	Centrado en el pipeline de compilación clásico. No aborda la evaluación de intérpretes ni los ambientes de ejecución funcionales.
Cai et al. (2023) [18]	Source Academy	Herramienta web que visualiza los ambientes de ejecución mediante representaciones gráficas de marcos y clausuras, argumentando que los modelos basados en pila no capturan correctamente la estructura de ambientes léxicos. Utilizada en un curso introductorio de CS en la Universidad Nacional de Singapur.	Diseñada para lenguajes predefinidos del proyecto Source Academy. No permite al estudiante definir su propia gramática ni visualizar el AST generado por un intérprete propio.

(continuación)

Autor / Año	Herramienta	Aporte principal	Limitación / Vacío
del Vado Vírveda (2023) [19]	ITT	Sistema de tutoría interactiva que visualiza el proceso de diseño de compiladores desde la implementación hacia la teoría, con ejercicios guiados y retroalimentación inmediata sobre análisis léxico, sintáctico y generación de código. Reporta experiencias de uso en cursos de la Universidad Complutense de Madrid.	Orientado al pipeline completo en lenguajes imperativos. No visualiza el ambiente de ejecución ni la evaluación de expresiones en intérpretes definidos por el propio estudiante.
Spigariol et al. (2023) [5]	Intérprete pedagógico funcional	Combina un intérprete de un lenguaje funcional con componentes gráficos que visualizan las estructuras intermedias del proceso de interpretación, permitiendo seguir el comportamiento del evaluador paso a paso. Es el antecedente de mayor cercanía pedagógica con el prototipo propuesto en este trabajo.	Específico para un lenguaje predefinido. No permite al estudiante definir su propia gramática ni trabajar con el modelo EOPL y SLLGEN en Racket.
Abad y Henz (2024) [20]	CSE Machine	Extiende el modelo de ambientes de SICP a una máquina nocional completa para Scheme que visualiza paso a paso los ambientes, las clausuras y el control de flujo. Utilizada en CS1101S de la Universidad Nacional de Singapur con retroalimentación positiva de estudiantes y tutores.	Integrada en el ecosistema cerrado de Source Academy para un lenguaje predefinido. No está diseñada para que el estudiante construya e inspeccione su propio intérprete EOPL con gramática definida en SLLGEN.
Stamenkovic y Jovanovic (2024) [3]	AWE	Sistema web educativo integral que combina visualización dinámica de estructuras de compilación, ejercicios guiados y experimentación directa para el aprendizaje del pipeline completo. Incluye módulos sobre análisis léxico y sintáctico, representación de árboles y generación de código. Reporta evidencia empírica de mejoras en comprensión con grupos de control y cuestionarios validados en instituciones de educación superior en Serbia.	Abarca el pipeline de compilación completo, pero no aborda la construcción interactiva de intérpretes EOPL ni la visualización del ambiente de ejecución desde la perspectiva del estudiante que implementa su propio evaluador.

Fuente: elaboración propia a partir de la revisión de la literatura.

A partir del análisis comparativo de los trabajos identificados se reconocen tres vacíos que fundamentan el desarrollo de una propuesta específica para el contexto de la asignatura FLP. En primer lugar, las herramientas que más se aproximan al objetivo aquí planteado (Cai et al., 2023; Abad y Henz, 2024; Spigariol et al., 2023) operan sobre lenguajes predefinidos integrados en ecosistemas ce-

rrados, sin posibilidad de que el estudiante defina su propia gramática ni intervenga en la estructura del evaluador. En segundo lugar, ninguna de las soluciones revisadas ofrece un flujo integrado que combine la edición de gramáticas BNF en notación SLLGEN, la visualización del AST resultante y la inspección del ambiente de ejecución dentro de un único entorno diseñado para acompañar el trabajo con la librería EOPL en Racket. En tercer lugar, la evidencia empírica en cursos de lenguajes de programación de programas de ingeniería en universidades latinoamericanas es escasa, lo que demanda adaptación y validación contextualizada [6, 7].

Con el propósito de atender estos vacíos, este trabajo desarrolla un prototipo web diseñado para acompañar el aprendizaje de la asignatura FLP de la Universidad del Valle. A diferencia de las herramientas revisadas, el prototipo propuesto permite al estudiante definir su propia gramática en notación BNF, genera automáticamente el código Racket necesario para que SLLGEN construya el analizador correspondiente y presenta de forma simultánea la estructura del AST producido junto con el estado del ambiente de ejecución tras cada evaluación. Esta integración busca reducir los cambios de contexto que el flujo de trabajo convencional con Racket y EOPL impone sobre el estudiante, concentrando en un único entorno las representaciones que se deben relacionar para comprender el proceso de interpretación [9].

Capítulo 3

Diagnóstico de dificultades conceptuales

Para identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura FLP, se realizó un diagnóstico basado en tres fuentes complementarias: una encuesta dirigida a estudiantes que han cursado la asignatura en distintos períodos académicos, una encuesta aplicada a docentes del programa de Ingeniería de Sistemas y una revisión del microcurrículo de la asignatura.

Estas fuentes permitieron obtener una caracterización de las dificultades desde diferentes perspectivas, teniendo la de los estudiantes que las experimentan directamente, la de los docentes que se encargan de enseñar el contenido y la del diseño curricular que estructura el aprendizaje, lo que generó una selección justificada de los conceptos que servirán como base para la biblioteca de casos predefinidos del prototipo.

3.1. Metodología

El diagnóstico se apoyó en un enfoque mixto, combinando métodos cuantitativos provenientes de encuestas con escala Likert y métodos cualitativos a través de preguntas abiertas y análisis documental del microcurrículo.

3.1.1. Diseño de los instrumentos

El diseño de ambas encuestas se fundamentó en las dimensiones de evaluación identificadas en la literatura acerca de las dificultades en la enseñanza de compiladores e intérpretes [3] [6] y sobrecarga cognitiva en programación [11], que orientaron la selección de los ítems a evaluar: comprensión teórica, capacidad de aplicación, percepción de las herramientas, disponibilidad de recursos y valoración de alternativas interactivas. Por otro lado, el microcurrículo oficial de la asignatura permitió anclar las preguntas a los resultados de aprendizaje e indicadores de logro concretos de FLP, evitando que el instrumento midiera percepciones genéricas desvinculadas del contexto específico de la asignatura.

La encuesta a estudiantes fue diseñada con siete ítems de escala Likert de cinco niveles (1: Totalmente en desacuerdo, 5: Totalmente de acuerdo) para evaluar la percepción de las dificultades conceptuales

en FLP, junto con una pregunta abierta orientada a identificar las recomendaciones de mejora. Esta estructura permitió distinguir entre dificultades atribuibles al contenido, al entorno de programación y a la escasez de recursos complementarios.

El diseño de la encuesta a docentes incluyó una pregunta de segmentación, teniendo en cuenta el semestre en el que cada docente dicta programación principalmente, seguida de bloques temáticos con ítems Likert sobre la percepción de las dificultades estudiantiles en distintos paradigmas de programación, cerrando con una pregunta abierta para comentarios o sugerencias respecto al proceso de enseñanza de programación.

Finalmente, la revisión del microcurrículo se realizó sobre el documento oficial de la asignatura FLP (código 750017C) el cual se encuentra en el Anexo A. Este documento estructura los contenidos a través de competencias específicas, resultados de aprendizaje (RA) e indicadores de logro (IL), lo que permitió contrastar las dificultades reportadas por estudiantes y docentes con la estructura formal de los contenidos e identificar cuáles indicadores concentran mayor complejidad conceptual.

3.1.2. Selección de la población

La población objetivo de la encuesta estudiantil fueron estudiantes que hubieran cursado FLP en la Universidad del Valle sede Tuluá durante los últimos tres años académicos, optando por no restringir la convocatoria al período en curso, con el objetivo de ampliar el tamaño de la muestra y capturar perspectivas recientes y en retrospectiva, lo que permitió que el diagnóstico reflejara una experiencia acumulada con la asignatura. La encuesta fue aplicada de forma voluntaria, obteniendo 36 respuestas adjuntas en el Anexo B.

La población objetivo de la encuesta docente fueron los profesores del programa de Ingeniería de Sistemas que dictan asignaturas de programación, independientemente del semestre. Esta decisión busca obtener una perspectiva longitudinal del programa, considerando que mientras los docentes de semestres avanzados informan sobre las dificultades que observan directamente en FLP y asignaturas afines, los docentes de primeros semestres permiten caracterizar las bases con las que los estudiantes ingresan. La encuesta fue aplicada de forma voluntaria, obteniendo 5 respuestas, las cuales se encuentran detalladas en el Anexo C.

3.2. Análisis de la encuesta a estudiantes

3.2.1. Resultados cuantitativos

La Tabla 3.1 presenta la distribución de respuestas para los siete ítems Likert de la encuesta aplicada a los 36 estudiantes.

Tabla 3.1: Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$)

Ítem	Tot. desac.	En desac.	Neutral	De acuerdo	Tot. acuerdo	% Acuerdo	% Desacuerdo
P1. Comprendí adecuadamente los conceptos teóricos	8.3 %	19.4 %	27.8 %	36.1 %	8.3 %	44.4 %	27.8 %
P2. Las clases magistrales fueron útiles	0.0 %	8.3 %	36.1 %	33.3 %	22.2 %	55.6 %	8.3 %
P3. Me siento preparado para aplicar los conceptos	8.3 %	36.1 %	36.1 %	19.4 %	0.0 %	19.4 %	44.4 %
P4. Tuve dificultades con conceptos abstractos	2.8 %	13.9 %	5.6 %	52.8 %	25.0 %	77.8 %	16.7 %
P5. Racket y EOPL facilitaron la comprensión	13.9 %	25.0 %	41.7 %	19.4 %	0.0 %	19.4 %	38.9 %
P6. Fue fácil encontrar recursos externos	16.7 %	44.4 %	22.2 %	5.6 %	11.1 %	16.7 %	61.1 %
P7. Recursos adicionales habrían mejorado mi aprendizaje	0.0 %	2.8 %	2.8 %	38.9 %	55.6 %	94.4 %	2.8 %

Fuente: elaboración propia.

Los resultados permiten identificar cuatro hallazgos clave:

Brecha entre comprensión teórica y aplicación práctica : Aunque el 44.4 % de los estudiantes se mostraron de acuerdo en haber comprendido adecuadamente los conceptos teóricos (P1), solo el 19.4 % se sintieron preparados para aplicarlos en otros cursos o proyectos (P3). Esta diferencia de puntos porcentuales revela que la comprensión teórica de los contenidos no se traduce necesariamente en una comprensión funcional que permita su uso autónomo. Adicionalmente, el porcentaje de respuestas neutrales en P3 (36.1 %) es significativamente mayor que en P1 (27.8 %), lo que sugiere que muchos estudiantes no tienen una opinión clara sobre su preparación para aplicar los conceptos, lo que refuerza la idea de una brecha entre teoría y práctica.

Alta incidencia de dificultades con conceptos abstractos : El ítem P4 concentra el mayor porcentaje de acuerdo de toda la encuesta, el 77.8 % de los participantes reportó haber tenido dificultades para comprender conceptos como entornos, estado o secuenciación. Este dato es especialmente significativo porque estos conceptos corresponden directamente al segundo resultado de aprendizaje (R.A.2) del microcurrículo de FLP, el cual concentra el 50 % del total de la nota de la asignatura.

Percepción negativa o neutra de Racket y EOPL como herramientas de aprendizaje : El ítem P5 muestra que solo el 19.4 % de los estudiantes consideró que Racket y EOPL facilitaron su comprensión, mientras que el 38.9 % estuvo en desacuerdo y el 41.7 % restante se mantuvo neutral. Este resultado sugiere que, para la mayoría de los participantes, las herramientas actuales no representan un apoyo claro al aprendizaje, lo que se alinea con la teoría de la carga cognitiva de Sweller, donde la sintaxis de Racket actúa como una carga extrínseca que satura la memoria de trabajo [9] y obstaculiza la comprensión semántica al obligar al estudiante a agotar sus recursos mentales en el descifrado visual del código [10].

Escasez de recursos externos y necesidad de herramientas adicionales : El 61.1 % de los estudiantes indicó que no fue fácil encontrar recursos externos de apoyo (P6), lo que apunta a un vacío en la disponibilidad de material complementario específico para los contenidos de la asignatura. Asimismo, el 94.4 % de los encuestados consideró que recursos adicionales habrían mejorado su aprendizaje de manera significativa (P7), constituyendo el dato de mayor consenso en toda la encuesta.

3.2.2. Resultados cualitativos

Del análisis temático de las recomendaciones proporcionadas por los estudiantes emergen cuatro categorías recurrentes:

Necesidad de experimentación con retroalimentación inmediata. Varios participantes señalaron que el principal obstáculo no es la complejidad conceptual en sí, sino la imposibilidad de explorar los conceptos en tiempo real con acompañamiento. Un estudiante sintetizó esta situación con precisión: *“me gustaría poder tener al profesor en vivo para una pregunta, porque las dudas surgen a medida que uno va haciendo y practicando [...] me quedó el vacío de un diagrama de ambientes”*. Otro participante expresó la misma necesidad de forma más directa: *“crear espacios donde los estudiantes puedan experimentar con Racket y la librería EOPL en tiempo real, con retroalimentación inmediata”*.

Carga cognitiva de Racket como obstáculo al aprendizaje semántico. La fricción generada por la sintaxis de Racket fue identificada de forma explícita por varios estudiantes como un factor que desplaza el foco del aprendizaje. El testimonio más detallado al respecto señala: *“el principal obstáculo del curso es la carga cognitiva que impone Racket. [...] Los nombres de funciones internas (como *car*, *cdr* o *map*) y la estructura visual plana terminan desplazando el foco de la semántica de lenguajes hacia la resolución de errores sintácticos”*.

Limitaciones de la metodología de aula invertida. Cuatro estudiantes mencionaron explícitamente la metodología de aula invertida como un factor de dificultad. Sus comentarios señalan que este modelo, si bien puede ser efectivo en otros contextos, resulta especialmente problemático para una asignatura con alto nivel de abstracción, donde la orientación directa del docente durante la exposición inicial de los conceptos es percibida como necesaria: *“con aula invertida es muy difícil entender”*; *“mayor explicación, resolver ejercicios desde cero con estudiantes”*.

Complejidad acumulada en los temas del último corte. Dos participantes señalaron que los temas de mayor dificultad se concentran en el tramo final del semestre, los cuales están relacionados con estado, paso de parámetros y tipos. Además, el ritmo de avance no permite profundizar en ellos adecuadamente: *“tomar más tiempo para la enseñanza de los temas más complejos que se ven en el último corte”*.

3.3. Análisis de la encuesta a docentes

La encuesta fue realizada por cinco docentes del programa identificados como D1 a D5. El instrumento segmentó los bloques temáticos, permitiendo que los cinco diligenciaran el bloque de percepción

general; D1 y D5 respondieron lo relacionado a la programación imperativa y programación orientada a objetos, teniendo en cuenta que dictan en primero y segundo semestre. Por otro lado, D2, D3 y D4 realizaron el bloque de programación lógica y paradigmas avanzados, lo que corresponde a quinto semestre en adelante. Si bien, el perfil directamente relacionado con FLP es el de los docentes de semestres avanzados, las respuestas de D1 y D5 permiten caracterizar las bases con las que los estudiantes llegan a la asignatura, lo que es un factor determinante en su capacidad de asimilar contenidos de alta abstracción [11].

3.3.1. Percepción general de los estudiantes

Este bloque obtuvo la respuesta de los cinco docentes y permite construir un diagnóstico transversal del programa. La Tabla 3.2 presenta los resultados, agrupando primero a los docentes de semestres avanzados (D2, D3, D4) y luego a los de primeros semestres (D1, D5), para facilitar la comparación por grupo.

Tabla 3.2: Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$)

Afirmación	D2 (5°+)	D3 (5°+)	D4 (5°+)	D1 (1°-2°)	D5 (1°-2°)
Demuestran habilidades básicas de pensamiento lógico	De acuerdo	Tot. acuerdo	Tot. acuerdo	Neutral	En desacuerdo
Tienen dificultades para analizar un problema antes de programar	En desacuerdo	De acuerdo	De acuerdo	En desacuerdo	Neutral
Logran depurar errores de manera efectiva	De acuerdo	De acuerdo	Neutral	En desacuerdo	Neutral
Pueden dividir problemas complejos en partes más pequeñas	Tot. acuerdo	Neutral	En desacuerdo	En desacuerdo	Neutral
Comprenden la importancia de la abstracción	Tot. acuerdo	Tot. acuerdo	En desacuerdo	En desacuerdo	Neutral

Fuente: elaboración propia.

La lectura de la Tabla 3.2 revela una brecha de percepción entre grupos. Los docentes de semestres avanzados tienden a valorar positivamente las habilidades de sus estudiantes más que los de primeros semestres, lo cual es un resultado esperable dado que los estudiantes de semestres superiores han superado los filtros académicos iniciales. D1 se muestra consistentemente en desacuerdo en cuatro de los cinco ítems, mientras que D5 se mantiene neutral, ambos describiendo un perfil de estudiante de primeros semestres con habilidades lógicas y de abstracción mínimas.

Además, se observa una variedad de respuestas dentro del grupo avanzado, mientras D2 y D3 coinciden en que los estudiantes de semestre superiores comprenden la importancia de la abstracción, D4 está en desacuerdo con esta afirmación. Lo mismo ocurre con la capacidad de dividir problemas complejos, donde D2 responde "Totalmente de acuerdo" y D4 "En desacuerdo". Estas diferencias dentro del mismo grupo de docentes indica que las habilidades de abstracción y descomposición de problemas no están consolidadas de manera uniforme en los estudiantes.

Esta observación es coherente con lo expuesto por Zagami, quien señala que los estudiantes enfrentan una alta demanda cognitiva al intentar procesar conceptos abstractos simultáneamente con habilidades prácticas; cuando la capacidad de la memoria de trabajo se ve excedida por esta multiplicidad de elementos, se generan bloqueos y sentimientos de frustración en el aprendiz [11]. Asimismo, se

reconoce que la naturaleza intrínsecamente abstracta de los algoritmos y construcciones teóricas en esta disciplina representa un desafío que dificulta la comprensión intuitiva [3].

3.3.2. Dificultades con paradigmas avanzados

Los docentes D2, D3 y D4 respondieron el bloque sobre los paradigmas directamente relacionados con los contenidos de FLP. La Tabla 3.3 muestra sus respuestas, evidenciando una percepción más positiva en D2 y D3, mientras que D4 se muestra más crítico, especialmente en relación con la capacidad de los estudiantes para seleccionar el paradigma adecuado para resolver un problema.

Tabla 3.3: Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$)

Afirmación	D2 (5°+)	D3 (5°+)	D4 (5°+)
Comprenden los conceptos fundamentales de programación lógica	Tot. acuerdo	Tot. acuerdo	Tot. acuerdo
Tienen dificultades con el razonamiento lógico aplicado a la programación	De acuerdo	De acuerdo	En desacuerdo
Logran modelar problemas usando reglas lógicas	Tot. acuerdo	Neutral	De acuerdo
Comprenden la diferencia entre los distintos paradigmas	Tot. acuerdo	De acuerdo	En desacuerdo
Pueden seleccionar el paradigma adecuado para resolver un problema	De acuerdo	De acuerdo	Tot. en desacuerdo

Fuente: elaboración propia.

El único ítem en el que los tres docentes coinciden es la comprensión de los conceptos fundamentales de programación lógica, donde todos respondieron "Totalmente de acuerdo". Sin embargo, esta percepción contrasta con el resto de las respuestas lo que sugiere que, saber qué es la programación lógica no implica saber aplicarla para resolver problemas.

Las opiniones también difieren respecto al razonamiento lógico aplicado a la programación. D2 y D3 consideran que los estudiantes presentan dificultades en este aspecto, mientras que D4 no lo percibe así. Estas diferencias pueden estar relacionadas con el contexto y el tipo de actividades realizados, ya que en cursos más guiados, algunas dificultades pueden pasar desapercibidas, pero, en asignaturas como FLP, donde los estudiantes deben construir soluciones con mayor autonomía, estas limitaciones resultan más evidentes.

Así mismo, la capacidad de comprender las diferencias entre paradigmas y seleccionar el adecuado para un problema concentra la mayor dispersión del bloque, debido a que D2 y D3 tienen una percepción positiva, pero, D4 está respectivamente "En desacuerdo" y "Totalmente en desacuerdo". Esta percepción negativa demuestra el salto cognitivo que representa abandonar el paradigma imperativo para adoptar el paradigma funcional [21], dificultad que se agrava cuando el lenguaje utilizado impone una carga sintáctica adicional.

3.4. Revisión del microcurrículo

El microcurrículo de la asignatura FLP organiza sus contenidos en torno a tres competencias específicas y una competencia genérica, articuladas a través de tres resultados de aprendizaje y veinte

indicadores de logro distribuidos con pesos diferenciados en las actividades de evaluación.

El **Resultado de Aprendizaje 1 (R.A.1)**, con un peso del 40 % sobre la nota total, abarca la construcción de datos recursivos mediante gramáticas BNF, el manejo del cálculo lambda, alcance de variables, implementación de tipos abstractos de datos y la construcción de árboles de sintaxis abstracta. Este R.A. sienta las bases formales sobre las que se construye todo el trabajo posterior de la asignatura.

El **Resultado de Aprendizaje 2 (R.A.2)**, con el mayor peso de la asignatura (50 %), se centra en la implementación de lenguajes con ambientes de variables, el recorrido del AST, la construcción de intérpretes básicos y la implementación de conceptos como condicionales, clausuras, estado y paso de parámetros. La profundidad de este resultado, sumado a su peso evaluativo, lo convierte en el eje central de la asignatura y en la fuente principal de dificultades reportadas por los estudiantes.

El **Resultado de Aprendizaje 3 (R.A.3)**, con un peso del 5 %, tiene un carácter más conceptual, puesto que se enfoca en que los estudiantes comprendan las diferencias entre compilación e interpretación identificando sus beneficios y limitaciones.

La **Competencia Genérica (C.G.2)**, con un peso del 5 %, integra varios indicadores del R.A.2 en la construcción de un intérprete completo evaluado en el proyecto final de la asignatura.

3.5. Selección de conceptos base para el prototipo

A partir de las dificultades reportadas por los estudiantes, las percepciones del cuerpo docente y la revisión del microcurrículo, se seleccionaron cinco indicadores de logro como base conceptual para la biblioteca de casos predefinidos del prototipo. El criterio de selección combinó tres factores, entre los que se encuentran el nivel de dificultad reportado por los estudiantes, el peso evaluativo en el microcurrículo y la viabilidad de representación visual e interactiva en una herramienta web.

La Tabla 3.4 presenta los indicadores de logro seleccionados, junto con su relación con los resultados de aprendizaje y una justificación para su inclusión en el prototipo.

Tabla 3.4: Indicadores de logro seleccionados como base para el prototipo

ID	Indicador de logro	R.A.	Justificación
IL 1.1.6	Construye un árbol de sintaxis abstracta a partir de una representación concreta en una gramática BNF	R.A.1	El AST es la estructura central del proceso de interpretación. Los estudiantes reportan dificultades para visualizar cómo una cadena de texto se transforma en una representación jerárquica; su visualización dinámica es la funcionalidad más distintiva del prototipo.
IL 1.2.3	Construye intérpretes básicos para dos conjuntos de valores (valor expresado, valor denotado)	R.A.2	Corresponde al primer nivel de construcción de intérpretes, donde se establece la relación entre la gramática, el AST y la evaluación. Es el punto de entrada natural para comprender el proceso completo de interpretación.
IL 1.2.6	Implementa lenguajes de programación con el concepto de estado asociado al manejo de ambientes para variables en lenguajes imperativos	R.A.2	El concepto de estado y su representación mediante ambientes fue identificado como el de mayor dificultad por los estudiantes (77.8 % reportó dificultades con entornos y estado). La representación visual de cómo evoluciona el ambiente durante la ejecución es uno de los aportes centrales del prototipo.
IL 1.2.7	Comprende la diferencia entre ligadura y asignación de variables	R.A.2	La distinción entre ligadura y asignación es conceptualmente sutil y frecuentemente confundida. Su representación visual mediante ambientes permite hacer explícita la diferencia que el código por sí solo no comunica con claridad.
IL 1.3.1	Identifica los componentes necesarios de un intérprete y un compilador	R.A.3	Provee el marco conceptual global que da sentido al resto de los contenidos. Comprender la arquitectura de un intérprete permite al estudiante situar cada concepto específico dentro de un proceso coherente.

Fuente: elaboración propia a partir del microcurrículo de la asignatura (código 750017C).

Estos cinco indicadores cubren los dos ejes de dificultad identificados en el diagnóstico. Por un lado, el eje sintáctico estructural, representado por IL 1.1.6, que corresponde a la construcción del AST. Por otro lado, el eje semántico-evaluativo, representado por IL 1.2.3, 1.2.6 y 1.2.7, referentes a la construcción del intérprete y el manejo del ambiente, complementado por el eje conceptual arquitectónico (IL 1.3.1).

Capítulo 4

Diseño del prototipo FLP Viewer

El presente capítulo documenta los resultados correspondientes al diseño del prototipo web denominado **FLP Viewer**, cuyo nombre combina la sigla de la asignatura con el término en inglés *viewer* (visualizador), que alude al propósito central de la herramienta: permitir al estudiante observar y explorar el comportamiento de un intérprete durante su ejecución.

En primer lugar, se presenta la especificación de requisitos del sistema, derivada del diagnóstico realizado en el capítulo anterior y de los indicadores de logro seleccionados como base conceptual del prototipo. En segundo lugar, se describe la arquitectura adoptada, la selección y justificación de tecnologías, los prototipos de interfaz de usuario y las estrategias de visualización implementadas para apoyar la comprensión de los contenidos de la asignatura.

4.1. Especificación de requisitos del sistema

4.1.1. Enfoque metodológico

La identificación y documentación de los requisitos del sistema se realizó siguiendo las directrices del estándar IEEE 830 [22] para la especificación de requisitos de software, adaptado al contexto educativo del proyecto. Los requisitos fueron derivados a partir de tres fuentes complementarias. En primer lugar, las dificultades conceptuales identificadas en el diagnóstico con estudiantes y docentes (Capítulo 3); en segundo lugar, los indicadores de logro del microcurrículo de la asignatura (Tabla 3.4); y en tercer lugar, las características de calidad de software definidas en la norma ISO/IEC 25010 [23], que orientó la formulación de los requisitos no funcionales.

Cada requisito fue clasificado por prioridad (Alta, Media) en función de su relevancia para los objetivos de aprendizaje identificados y de su viabilidad de implementación en el alcance del proyecto. Los criterios de aceptación asociados a cada requisito fueron formulados de manera verificable, haciendo referencia al comportamiento concreto del sistema implementado. La especificación completa se encuentra en el Anexo E.

4.1.2. Requisitos funcionales

Se definieron ocho requisitos funcionales que cubren las capacidades esenciales del prototipo. La Tabla 4.1 presenta el resumen de los mismos con su prioridad y una descripción concisa.

Tabla 4.1: Resumen de requisitos funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RF-01	Generación del AST	Procesar el programa del usuario con la gramática definida y producir el AST correspondiente.	Alta
RF-02	Visualización del AST	Renderizar el AST generado de forma jerárquica e interactiva en un panel dedicado.	Alta
RF-03	Ejecución y evaluación	Invocar el intérprete Racket sobre los archivos del editor y mostrar el resultado en la consola.	Alta
RF-04	Representación del ambiente	Mostrar el ambiente de evaluación como estructura jerárquica de frames actualizada tras cada ejecución.	Alta
RF-05	Biblioteca de ejemplos	Ofrecer un conjunto de programas predefinidos alineados con los indicadores de logro de la asignatura.	Media
RF-06	Validación de sintaxis Racket	Detectar y reportar errores léxicos y sintácticos en la gramática BNF y en el programa del usuario.	Alta
RF-07	Visualización del intérprete	Presentar los archivos del intérprete generado (gramática, ambiente, evaluador) en un editor organizado por pestañas con secciones protegidas.	Media
RF-08	Plantilla base y exportación	Proveer una plantilla editable del intérprete con secciones bloqueadas y permitir la descarga del proyecto en formato .zip.	Alta

Fuente: elaboración propia. Especificación completa en Anexo E.

4.1.3. Requisitos no funcionales

Se definieron cinco requisitos no funcionales que establecen las condiciones de calidad del sistema. La Tabla 4.2 presenta el resumen correspondiente.

Tabla 4.2: Resumen de requisitos no funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RNF-01	Usabilidad	La interfaz debe ser operada sin instrucción previa por estudiantes sin experiencia avanzada en compiladores.	Alta
RNF-02	Rendimiento	La generación del AST y la evaluación del programa deben completarse en menos de 5 segundos en condiciones normales.	Alta
RNF-03	Accesibilidad web	El sistema debe ejecutarse en navegadores modernos sin instalación adicional ni autenticación.	Alta
RNF-04	Mantenibilidad	El código debe estar organizado en módulos independientes con separación explícita de tipos, lógica y presentación.	Media
RNF-05	Escalabilidad	La arquitectura debe permitir incorporar nuevos ejemplos, gramáticas o características sin afectar las funcionalidades existentes.	Media

Fuente: elaboración propia. Especificación completa en Anexo E.

4.2. Diseño del prototipo

4.2.1. Arquitectura del sistema

La arquitectura del prototipo se estructuró en tres capas principales: infraestructura de despliegue, servidor y cliente. Esta separación de responsabilidades permite que el procesamiento computacionalmente intensivo (la ejecución de Racket) ocurra en el servidor, mientras que la interfaz de usuario opera íntegramente en el navegador sin necesidad de recargas de página [24].

Las decisiones que dieron lugar a esta arquitectura estuvieron guiadas por tres criterios principales: simplicidad operativa, aislamiento de la ejecución de código arbitrario y minimización de la latencia percibida por el usuario. A partir de estos criterios se definió la distribución de responsabilidades entre los componentes y el modelo de despliegue del sistema.

Con respecto a la simplicidad operativa, se optó por una arquitectura monolítica de servidor único en lugar de una arquitectura de microservicios. Fowler y Lewis señalan que los microservicios introducen una complejidad operativa significativa, relacionada con la coordinación entre servicios, la trazabilidad distribuida y la gestión de fallos, que solo se justifica cuando la escala o la independencia de despliegue de los componentes lo requieren [25]. Dado que el prototipo cuenta con un equipo de desarrollo reducido y un conjunto acotado de funcionalidades, la arquitectura monolítica resulta más adecuada para el alcance del proyecto.

Respecto al aislamiento de la ejecución de código arbitrario, la ejecución de Racket se delega a un subproceso del servidor de aplicación en lugar de separarse como un servicio independiente. Cada invocación se ejecuta en un proceso con un tiempo límite estricto y un directorio temporal propio. De esta manera, un programa con un bucle infinito o un comportamiento inesperado no compromete la estabilidad del servidor principal [26].

Finalmente, para reducir la latencia percibida por el usuario, el pipeline de transformación de gramática se ubicó íntegramente en el cliente. Esta decisión acerca el cómputo al punto de generación de los datos y elimina la latencia de red asociada a las validaciones inmediatas [24]. Además, esta estrategia es viable porque la carga computacional del pipeline es baja y no requiere acceso a recursos

del servidor.

4.2.1.1. Infraestructura de despliegue

La infraestructura de despliegue se compone de dos elementos. El primero es un reverse proxy¹, encargado de recibir las solicitudes entrantes y redirigirlas al servidor de aplicación, centralizando así el punto de entrada al sistema. El segundo es el propio servidor de aplicación, que incorpora el intérprete de Racket como dependencia del entorno de ejecución. Esta configuración garantiza que el prototipo sea reproducible independientemente del sistema operativo del servidor anfitrión. La selección de las tecnologías concretas que materializan esta estructura se detalla en la sección 4.2.2.

4.2.1.2. Capa de servidor

La capa de servidor asume dos responsabilidades principales: el renderizado inicial de la interfaz y la ejecución del intérprete. Durante el renderizado inicial, los datos estáticos del sistema, como los ejemplos predefinidos y los contenidos de ayuda, se cargan desde el sistema de archivos y se transmiten directamente a la capa cliente. De esta manera, se evita la necesidad de realizar peticiones adicionales una vez cargada la página.

La segunda responsabilidad corresponde a la ejecución de programas en Racket. Para ello, el servidor expone un punto de acceso interno que recibe los archivos generados por el editor y la expresión a evaluar. A continuación, lanza el proceso de Racket de forma aislada, aplicando un tiempo límite configurable para su ejecución, y devuelve al cliente el resultado estructurado como una secuencia de pasos.

4.2.1.3. Capa de cliente

La capa cliente se diseñó alrededor de un componente orquestador central encargado de mantener el estado global de la sesión de trabajo y coordinar la comunicación entre los distintos paneles de la interfaz. Este componente centraliza las transiciones de estado, incluyendo el inicio y fin de sesión, la recepción de resultados y la navegación paso a paso por las evaluaciones. Gracias a ello, cada panel de visualización puede implementarse como un componente stateless² que recibe únicamente la información necesaria para su renderizado.

¹Un *reverse proxy* es un servidor interpuesto entre el cliente y el servidor de aplicación que recibe las solicitudes entrantes y las reenvía al servicio interno correspondiente, ocultando los detalles de la infraestructura backend y permitiendo aplicar reglas de seguridad, balanceo de carga o compresión de manera centralizada.

²Un componente *stateless* (sin estado) es aquel que no almacena información propia entre renders; su salida depende exclusivamente de los datos que recibe como entrada, lo que facilita su prueba y reutilización.

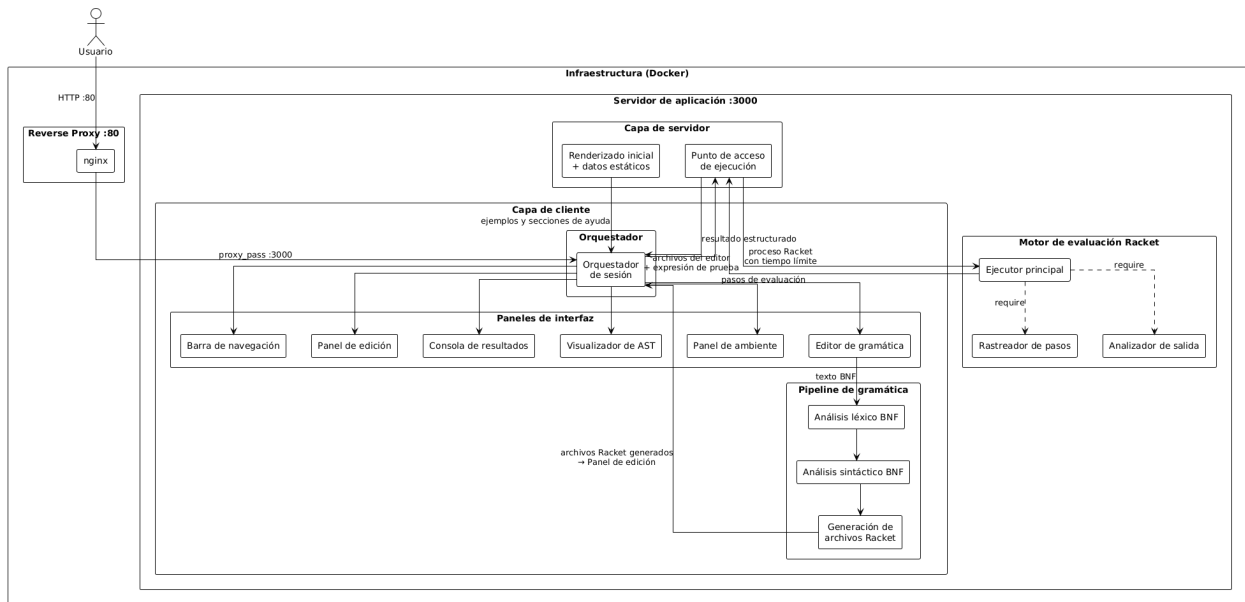


Figura 4.1: Diagrama de arquitectura del sistema

Fuente: Elaboración propia

4.2.1.4. Pipeline de gramática

Dentro de la arquitectura cliente, el pipeline de transformación de gramática constituye un subsistema encargado de procesar las especificaciones definidas por el estudiante sin intervención del servidor. Se diseñó como una cadena de etapas independientes que comprende el análisis léxico de la notación BNF, el análisis sintáctico para la construcción de una representación intermedia de la gramática y la generación de los archivos del intérprete a partir de dicha representación.

La separación en etapas independientes permite identificar y reportar errores de forma precisa en el nivel donde se originan (léxico o sintáctico) y favorece una organización modular del proceso de transformación [8]. Además, esta estructura facilita la incorporación de nuevas etapas o transformaciones sin alterar significativamente los componentes existentes.

Al ejecutarse íntegramente en el cliente, el pipeline proporciona retroalimentación inmediata sobre errores en la gramática sin depender de intercambios con el servidor. Esta característica reduce el tiempo de espera entre la acción del estudiante y la respuesta del sistema, favorece la continuidad de la tarea y contribuye a disminuir la carga cognitiva extrínseca [9].

4.2.2. Selección de tecnologías

La selección de tecnologías se realizó evaluando cuatro criterios. Se consideraron la curva de aprendizaje del equipo de desarrollo, la disponibilidad de documentación, la viabilidad de implementación de las funcionalidades requeridas y la compatibilidad entre las tecnologías seleccionadas. La Tabla 4.3 presenta la justificación de cada elección.

Tabla 4.3: Tecnologías seleccionadas y justificación

Tecnología	Versión	Justificación
Next.js [27]	16.2.6	Framework basado en React que integra renderizado en servidor y rutas de API en una única plataforma. Su adopción permite implementar la arquitectura propuesta sin requerir servicios backend adicionales.
React [28]	19.2	Biblioteca para la construcción de interfaces de usuario basada en componentes. Su modelo declarativo facilita la implementación de la interfaz interactiva y la separación de responsabilidades entre los distintos paneles del sistema.
TypeScript [29]	5.9	Superset tipado de JavaScript que permite definir contratos explícitos entre módulos mediante interfaces y tipos. Su utilización contribuye a reducir errores durante el desarrollo y a mejorar la mantenibilidad del código.
Monaco Editor [30]	0.55	Componente de edición utilizado por Visual Studio Code y disponible como librería independiente. Proporciona las capacidades necesarias para implementar el editor interactivo, incluyendo la protección de regiones de código generadas automáticamente.
Tailwind CSS [31]	4	Framework de estilos utilitarios que facilita la construcción de interfaces consistentes mediante clases reutilizables. Su enfoque permite mantener los estilos próximos a los componentes que los utilizan y simplifica el desarrollo de la capa de presentación.
Racket [4]	9.x	Lenguaje utilizado en la asignatura FLP y plataforma de ejecución de la librería EOPL. Su adopción garantiza la compatibilidad entre el código generado por el prototipo y el entorno académico empleado por los estudiantes.
Docker [32]	—	Plataforma de contenedores utilizada para encapsular el entorno de ejecución. Permite garantizar la reproducibilidad del despliegue e incorporar Racket como dependencia sin requerir su instalación en el sistema anfitrión.
nginx	alpine	Servidor web empleado como reverse proxy delante del servidor de aplicación. Centraliza el acceso al sistema y permite gestionar aspectos de infraestructura como compresión, redirección de solicitudes y configuración de seguridad.
pnpm	10	Gestor de paquetes para el ecosistema JavaScript con soporte de almacenamiento eficiente de dependencias. Su utilización reduce los tiempos de instalación y construcción del proyecto en el entorno de despliegue.

Fuente: elaboración propia.

La selección realizada refleja las decisiones arquitectónicas descritas en la sección anterior. Next.js, React y TypeScript conforman la base de la capa cliente y del servidor de aplicación, mientras que Monaco Editor proporciona las capacidades de edición requeridas por el entorno interactivo. Por su parte, Racket constituye el motor de evaluación del prototipo y Docker garantiza la reproducibilidad de su entorno de ejecución. Finalmente, nginx complementa la infraestructura de despliegue como punto de entrada único al sistema.

4.2.3. Prototipos de interfaz de usuario

La distribución de la interfaz se diseñó con base en los principios de economía de la atención [16] y en los hallazgos del diagnóstico, que evidenciaron que la fricción visual de las herramientas actuales

desplaza la atención del estudiante desde la semántica del lenguaje hacia la resolución de errores sintácticos. Para mitigar este efecto, la interfaz se organizó de manera que las tareas principales del estudiante, que incluyen la definición de la gramática, la edición del intérprete, la ejecución de programas y el análisis de resultados, siguieran un recorrido visual coherente dentro de una única pantalla, minimizando los cambios de contexto.

La literatura sobre herramientas educativas para la enseñanza de compiladores e intérpretes destaca la importancia de presentar simultáneamente múltiples representaciones del proceso de ejecución, permitiendo que el estudiante relacione el código fuente con las estructuras internas generadas durante el procesamiento [3,5,6]. Asimismo, los principios de carga cognitiva sugieren que la integración de información relevante dentro de un mismo espacio de trabajo reduce el esfuerzo necesario para coordinar fuentes dispersas de información [9]. Por esta razón, se optó por una interfaz multipanel en la que las representaciones principales del proceso de interpretación permanecen visibles de manera simultánea.

La interfaz principal se estructura en cinco zonas funcionales, cuya distribución se presenta en el wireframe³ de la Figura 4.2. La organización propuesta define una configuración inicial de los paneles, permitiendo al estudiante modificar dinámicamente el espacio asignado a cada zona durante la interacción. De esta manera, es posible redistribuir el área disponible entre el editor, las visualizaciones del AST y del ambiente, y la consola de salida según las necesidades de la tarea en curso. Esta flexibilidad permite priorizar las representaciones más relevantes en cada momento, manteniendo una visión integrada del proceso de interpretación y contribuyendo a la usabilidad del sistema [7].

1. **Barra de navegación** (zona superior): concentra el acceso a las funciones globales del sistema, incluyendo la biblioteca de ejemplos, el editor de gramática y los controles asociados a la sesión de ejecución.
2. **Panel de edición** (zona izquierda): alberga el editor de código organizado en múltiples pestañas, desde donde el estudiante interactúa con los distintos archivos que conforman el intérprete.
3. **Panel del AST** (zona derecha superior): presenta la representación jerárquica del árbol de sintaxis abstracta generado durante la evaluación de expresiones.
4. **Panel de ambiente** (zona derecha inferior): visualiza el estado del entorno de ejecución mediante una estructura de frames anidados que muestra los bindings activos en cada nivel de alcance.
5. **Panel de consola** (zona inferior): concentra la interacción textual con el intérprete, permitiendo introducir expresiones y consultar los resultados producidos durante la ejecución.

³Un *wireframe* es un prototipo de baja fidelidad utilizado en el diseño de interfaces para representar la estructura, distribución espacial y flujo de interacción de una aplicación, sin incorporar elementos visuales finales ni detalles de implementación.

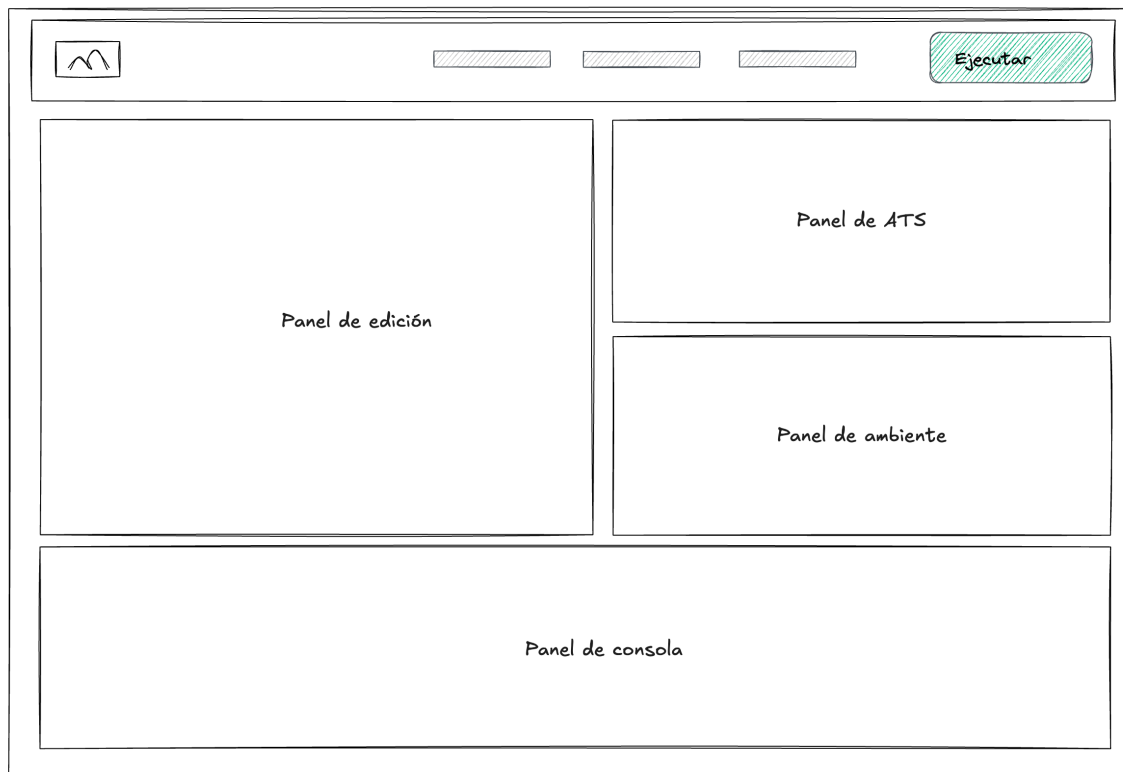


Figura 4.2: Wireframe de la interfaz principal del prototipo FLP Viewer

Fuente: Elaboración propia

Adicionalmente, se diseñó un modal de gramática (Figura 4.3) que permite al estudiante ingresar la especificación léxica y la gramática en notación BNF en dos áreas de texto diferenciadas. Al confirmar la definición, el sistema genera los archivos del intérprete y los carga en el panel de edición. Este flujo de interacción mantiene la gramática y el intérprete en un vínculo explícito y visible para el estudiante, de forma que la relación entre la especificación formal y el código generado pueda observarse directamente. Esta decisión responde a la necesidad de hacer visibles procesos que normalmente permanecen ocultos en los entornos de desarrollo convencionales, una estrategia identificada como beneficiosa para la enseñanza de compiladores e intérpretes [3, 5].

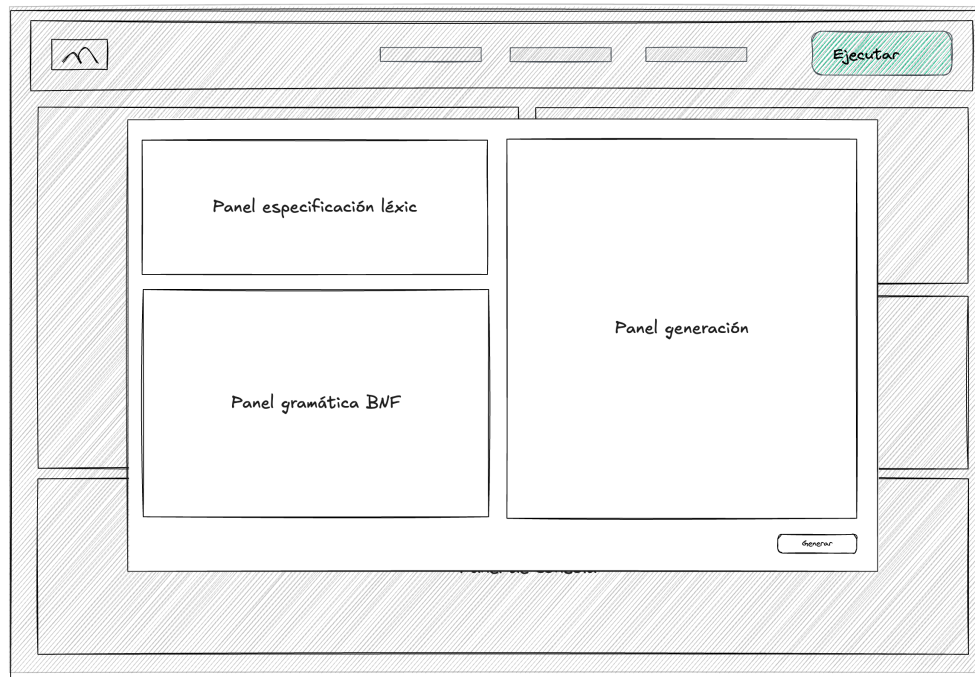


Figura 4.3: Wireframe del modal de definición de gramática BNF

Fuente: Elaboración propia

4.2.4. Estrategias de visualización

El diseño de las estrategias de visualización responde directamente a los dos ejes de dificultad identificados en el diagnóstico: el eje sintáctico-estructural, asociado a la comprensión del AST, y el eje semántico-evaluativo, asociado al seguimiento del estado durante la ejecución [6]. En cada caso, las decisiones de diseño se orientaron a maximizar la correspondencia entre las representaciones visuales y los modelos conceptuales presentados en la asignatura.

El AST se renderiza como un árbol jerárquico interactivo en el que cada nodo muestra el nombre del tipo de producción que lo originó y, cuando corresponde, el valor léxico del token asociado. Esta representación hace visible la correspondencia entre la notación BNF definida por el estudiante y la estructura interna que el parser produce. Sorva et al. señalan que la efectividad de este tipo de herramientas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [13]. Para apoyar esa lectura, los nodos se diferencian visualmente según su categoría semántica, lo que es consistente con los hallazgos de Stamenkovic et al. [3] y Jovanovic et al. [6]. Asimismo, los nodos admiten expansión y colapso interactivo y, por defecto, solo el nivel raíz se expande al cargar un nuevo árbol, de modo que el estudiante parte de una vista de alto nivel y profundiza progresivamente según su interés, en coherencia con los principios de carga cognitiva [9].

El ambiente de ejecución se representa como una cadena de frames enlazados, donde cada frame expone las ligaduras activas en ese nivel de alcance con su nombre, valor y tipo. La disposición hace visible la relación de extensión entre los entornos, de modo que el estudiante puede seguir la cadena desde el entorno inicial hasta el más reciente. Esta representación busca hacer observable

la diferencia entre ligadura y asignación (IL 1.2.7) y la evolución del ambiente con cada llamada a función (IL 1.2.6), abordando directamente la dificultad reportada por el 77.8 % de los estudiantes encuestados. La disposición en cadena sigue el modelo conceptual descrito por Friedman y Wand [4]. Cai et al. argumentan que los modelos de visualización basados en pila no capturan correctamente la estructura de los ambientes léxicos en lenguajes modernos y que se requieren representaciones que reflejen el encadenamiento real de los frames [18]; la representación adoptada en FLP Viewer responde a esta misma consideración. Favorece también la correspondencia entre la teoría presentada en clase y la visualización ofrecida por la herramienta, lo cual es relevante porque representaciones que no coinciden con el modelo enseñado pueden inducir concepciones erróneas difíciles de corregir [14].

El modo de evaluación paso a paso permite que, cuando el estudiante envía múltiples expresiones al intérprete, pueda navegar por los resultados de cada una de forma individual. En cada paso, el AST y el ambiente se actualizan para reflejar el estado del intérprete tras esa evaluación, haciendo visible cómo evolucionan el árbol y las ligaduras de una expresión a la siguiente. Este mecanismo es análogo al *stepper* algebraico propuesto por Clements, Flatt y Felleisen [15] y sigue el enfoque de herramientas como Python Tutor [14] y la máquina nocional CSE de Abad y Henz [20]. Desde la perspectiva de la carga cognitiva, la presentación controlada de un paso a la vez reduce la presión sobre la memoria de trabajo [9].

Finalmente, la consola de salida diferencia mediante color y prefijo textual cinco categorías de mensaje: entrada del usuario, valor de retorno, información, advertencia y error. Esta distinción permite al estudiante identificar la naturaleza de cada mensaje sin necesidad de leer su contenido, reduciendo el esfuerzo de interpretación de la retroalimentación. La codificación cromática está respaldada por Hannebauer et al. [17], y la decisión de no exponer directamente la salida nativa de Racket responde a los principios de usabilidad en tecnología educativa que recomiendan facilitar la interpretación de la información relevante durante la interacción [7].

Capítulo 5

Implementación del prototipo FLP Viewer

El presente capítulo documenta los resultados correspondientes al desarrollo e implementación del prototipo FLP Viewer, en cumplimiento del objetivo específico 2 del proyecto. El capítulo se organiza en cinco secciones. En primer lugar, se describe el repositorio de código fuente y su estructura de directorios. En segundo lugar, se detallan los componentes implementados, abordando el pipeline de gramática, la arquitectura del cliente, el punto de acceso de ejecución y la instrumentación en tiempo de ejecución. En tercer lugar, se presenta el flujo principal de ejecución a través de un diagrama de secuencia. En cuarto lugar, se describe la configuración de despliegue en el entorno de pruebas. Finalmente, se verifica el cumplimiento de los requisitos funcionales definidos en el diseño.

5.1. Repositorio de código fuente

El código fuente del prototipo se encuentra alojado en el repositorio público de GitHub disponible en <https://github.com/Esmeralda-RG/flp-viewer>. El repositorio registra el historial completo de *commits* del proceso de desarrollo, organizado en torno a las funcionalidades incrementales del sistema. El archivo `README.md` incluye la descripción del proyecto, el stack tecnológico utilizado, las instrucciones de instalación y ejecución local, la estrategia de integración con Racket y la descripción de la estructura de directorios.

La estructura del proyecto sigue la convención del *App Router*¹ de Next.js [27], con los siguientes directorios principales:

- `app/api/run/`: contiene la ruta de API encargada de gestionar la ejecución del intérprete Racket, junto con los archivos `.rkt` de instrumentación en tiempo de ejecución (`_runner.rkt`, `_tracking.rkt` y `_stream-parser.rkt`).
- `app/components/`: agrupa los componentes de la interfaz de usuario, entre ellos el orquestador `PlaygroundLayout`, el visualizador de árbol `ASTViewer`, el panel de ambiente `EnvironmentPanel`,

¹El *App Router* es el sistema de enrutamiento de Next.js basado en el sistema de archivos introducido en la versión 13. Organiza las rutas, los componentes de servidor y la lógica de la API dentro del directorio `app/`, permitiendo mezclar componentes de servidor y de cliente en la misma aplicación.

la consola `ConsoleOutput`, el editor `EditorPanel` y los modales auxiliares.

- `app/lib/`: contiene los módulos del pipeline de gramática, los generadores de código Racket y las utilidades del playground.
- `app/services/`: incluye el cliente HTTP que comunica la capa cliente con la ruta de API.
- `app/types/`: centraliza las definiciones de tipos TypeScript compartidas entre módulos.
- `examples/`: almacena las plantillas de programas predefinidos cargadas en el renderizado inicial del servidor.

5.2. Implementación de los componentes

5.2.1. Pipeline de gramática BNF

El pipeline de gramática se implementó íntegramente en TypeScript [29] como una cadena de módulos independientes que opera en el cliente sin intervención del servidor. Su punto de entrada único es la función `runPipeline`, definida en `grammar-pipeline.ts`, que recibe la especificación léxica y la gramática en notación BNF y retorna los archivos del intérprete generados o una lista de errores.

La cadena de transformación sigue el modelo clásico de compilación [8] y comprende tres etapas secuenciales. La primera es el análisis léxico, implementado en `bnf-lexer.ts`: la función `tokenize` recorre el texto de entrada carácter a carácter y produce una secuencia de tokens tipados. Los errores léxicos se reportan mediante la clase `LexError`, que incluye la línea y columna exactas del fallo. La segunda etapa es el análisis sintáctico, implementado en `bnf-parser.ts`: la función `parse` consume la secuencia de tokens y construye una representación intermedia de la gramática (`GrammarAST`) compuesta por reglas de producción. Los errores sintácticos se reportan mediante la clase `ParseError` con la misma precisión posicional. La tercera etapa es la generación de código, distribuida en tres módulos especializados. `eopl-generator.ts` produce el archivo `grammar.rkt` con la especificación léxica y gramatical en formato SLLGEN². `env-generator.ts` genera `environment.rkt` con el ambiente inicial y las funciones de extensión. `main-generator.ts` produce `main.rkt` con los *stubs*³ de las funciones de evaluación pendientes de implementación por parte del estudiante, registrando además las líneas bloqueadas para protección en el editor.

La especificación léxica se define en un área de texto independiente del modal de gramática, escribiendo un nombre de token por línea. Para cada nombre reconocido (`number`, `float`, `identifier`, `binary`, `octal`, `hex` y `text`) la función `expandLexRule` de `eopl-generator.ts` produce las reglas SLLGEN correspondientes. Las reglas de espaciado (`whitespace`) y de comentarios de línea (`comment`) se incluyen siempre de forma automática. Si el área léxica se deja vacía, el pipeline incorpora el conjunto completo de tokens por defecto. Para tipos léxicos no contemplados en los nombres predefinidos, el estudiante puede escribir directamente la regla en notación SLLGEN entre paréntesis, que el pipeline incorpora sin transformaciones.

²SLLGEN es la biblioteca de análisis sintáctico incluida en EOPL que genera automáticamente un escáner y un analizador a partir de una especificación léxica y una gramática expresadas como listas Racket [4].

³Un *stub* es una función con cuerpo incompleto que declara la firma esperada y sirve como marcador del trabajo pendiente de implementación. En este contexto, los *stubs* definen las funciones de evaluación del intérprete que el estudiante debe completar.

5.2.2. Componente orquestador y paneles de la interfaz

La Figura 5.1 muestra el modal de bienvenida que se presenta al estudiante en el primer acceso. El modal ofrece un resumen de las acciones básicas y carga automáticamente el ejemplo *Hola Mundo* como punto de partida.

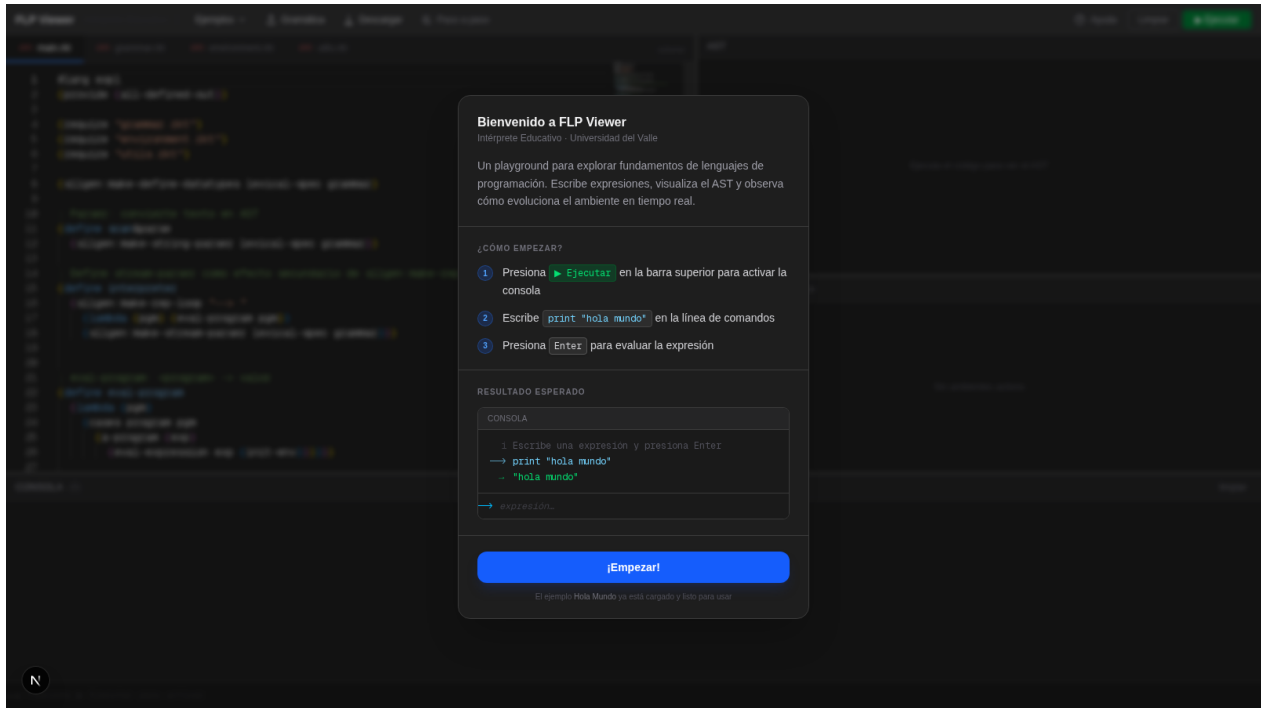


Figura 5.1: Modal de bienvenida del prototipo FLP Viewer

Fuente: Elaboración propia

El componente **PlaygroundLayout** actúa como orquestador central de la interfaz. Mantiene el estado global de la sesión de trabajo e implementa las transiciones de estado definidas en el diseño: inicio y fin de ejecución, recepción de resultados, avance paso a paso y apertura de modales auxiliares.

El estado que gestiona el orquestador comprende los archivos del editor (**files**), el árbol de sintaxis abstracta activo (**ast**), los marcos del ambiente de ejecución (**frames**), el historial de mensajes de la consola (**logs**), los indicadores de sesión activa y ejecución en curso, el modo de evaluación paso a paso (**stepMode**) y la cola de pasos pendientes (**pendingSteps**). Los cuatro paneles de visualización (**ASTViewer**, **EnvironmentPanel**, **ConsoleOutput** y **EditorPanel**) son componentes React [28] sin estado propio: reciben exclusivamente los datos necesarios para su renderizado a través de las propiedades que el orquestador les suministra.

El modo de evaluación paso a paso opera mediante una cola de pasos (**pendingSteps**) que se inicializa con todos los pasos retornados por la API. Cada vez que el usuario activa el control *Next step*, el orquestador extrae el siguiente paso de la cola y actualiza de forma sincronizada el árbol AST, los marcos del ambiente y la salida de la consola, permitiendo al estudiante observar la evolución del estado del intérprete expresión por expresión, en correspondencia con el modelo de evaluación por pasos propuesto por Clements et al. [15].

La Figura 5.2 ilustra la interfaz en modo paso a paso, con el botón de avance visible en la consola y el estado del AST y el ambiente correspondiente al paso activo.

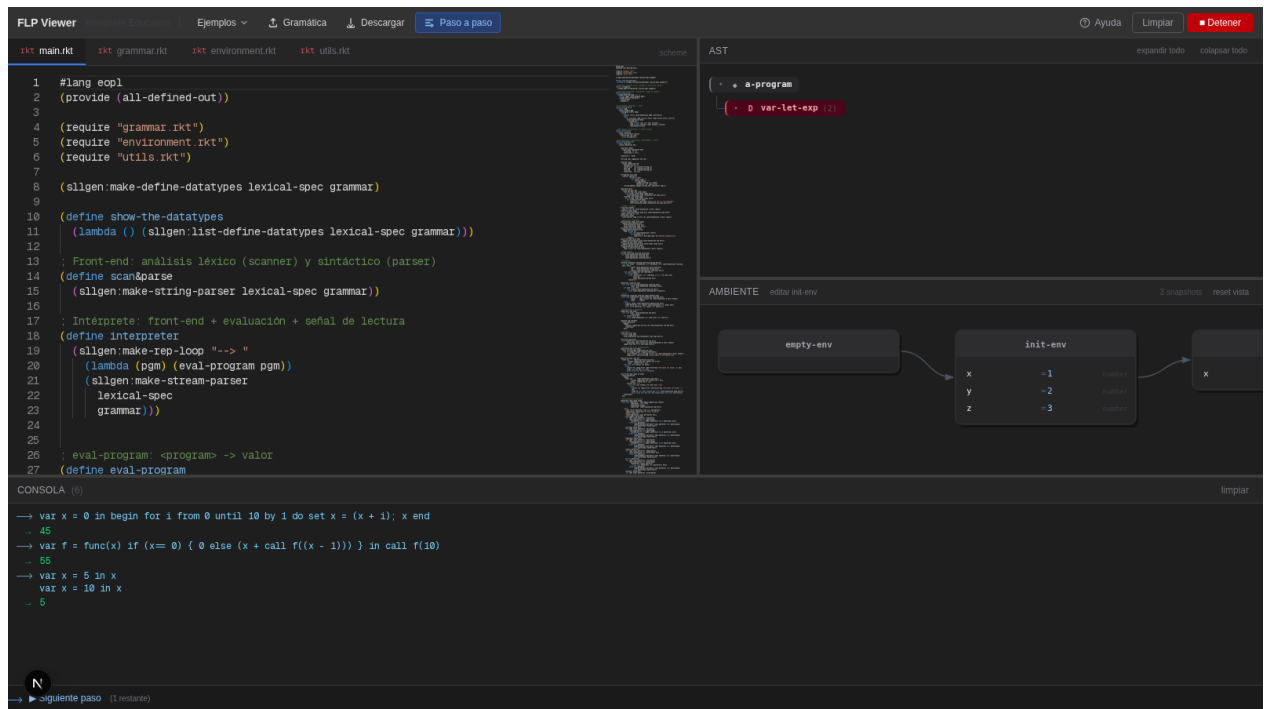


Figura 5.2: Modo de evaluación paso a paso con pasos pendientes de navegación

Fuente: Elaboración propia

La Figura 5.3 ilustra el modal de definición de gramática con una especificación BNF y la vista previa del archivo `grammar.rkt` generado.

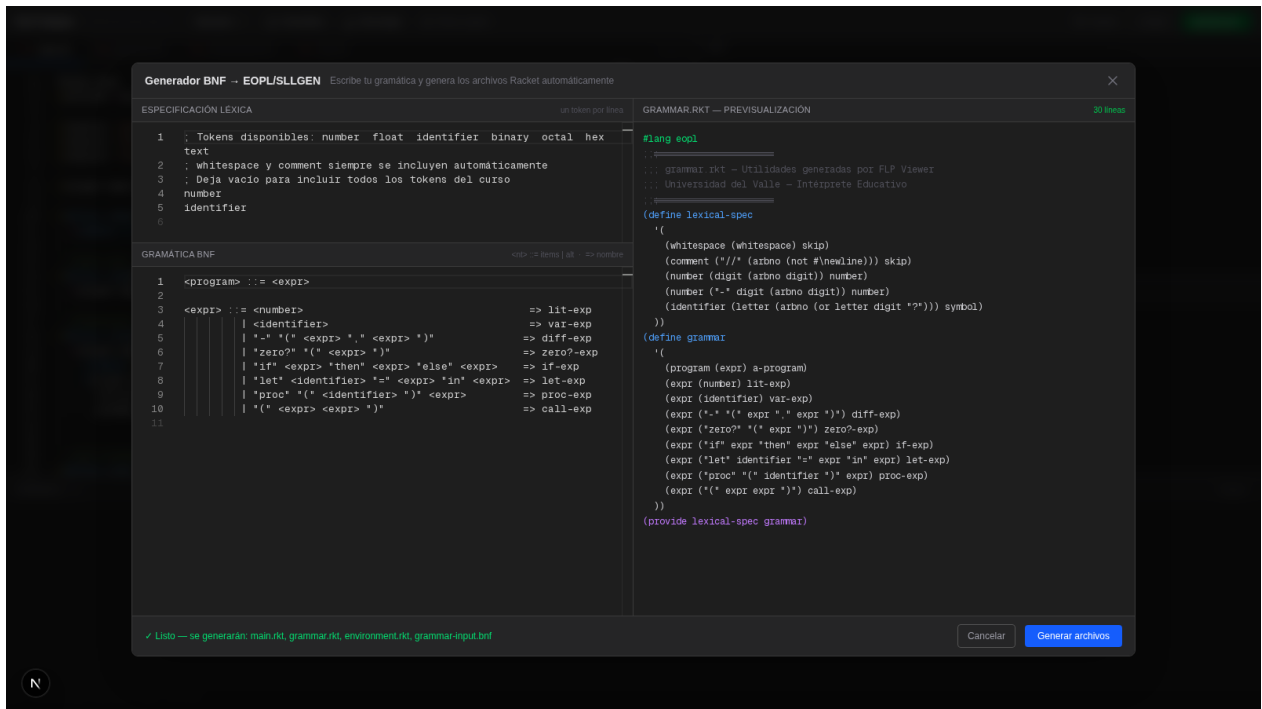


Figura 5.3: Modal de definición de gramática BNF con vista previa del archivo generado

Fuente: Elaboración propia

El modal de gramática (**GrammarModal**) recibe las entradas del estudiante y, al confirmar, invoca `runPipeline` para obtener los archivos generados, que son cargados en el panel de edición a través del mecanismo `upsertFile`. Si el pipeline reporta errores léxicos o sintácticos, estos se presentan en el modal antes de cerrar, sin modificar los archivos activos en el editor.

El componente **EditorPanel** utiliza Monaco Editor [30] para la edición de código. Las líneas generadas automáticamente por el pipeline se marcan como de solo lectura mediante la API de rangos de Monaco, de modo que el estudiante solo puede editar las secciones correspondientes a la lógica de evaluación.

Las Figuras 5.4 y 5.5 documentan dos evaluaciones representativas sobre el ejemplo *Intérprete EOPL (avanzado)*, seleccionadas por su correspondencia directa con los indicadores de logro IL 1.2.6 e IL 1.2.7, identificados en el Capítulo 3 como las fuentes de mayor dificultad conceptual del R.A.2 (Tabla 3.4). Dicho resultado concentra el 50 % del peso evaluativo de la asignatura, y el 77.8 % de los estudiantes encuestados reportó haber tenido dificultades con los conceptos asociados (entornos, estado y secuenciación) durante su cursada (P4, Tabla 3.1). Las evaluaciones cubren, de forma complementaria, los dos ejes semánticos que el diagnóstico identificó como prioritarios: el estado mutable con iteración, cuya comprensión exige distinguir entre ligadura y asignación destructiva (IL 1.2.7), y las funciones de orden superior con clausuras, cuya representación hace visible cómo el ambiente registra procedimientos como valores de primera clase almacenados junto a cualquier otra ligadura (IL 1.2.6) [4].

La Figura 5.4 evalúa la expresión `var x = 0 in begin for i from 0 until 10 by 1 do set x = (x + i); x end`, que acumula la suma de 0 a 9 mediante un bucle con asignación destructiva.

El panel de ambiente muestra la cadena `empty-env` $\rightarrow$ `init-env` $\rightarrow$ `x`, donde el marco final refleja el valor de `x` tras la última iteración del bucle. Este caso documenta que la instrumentación captura correctamente la evolución del estado mutable a lo largo de la ejecución.

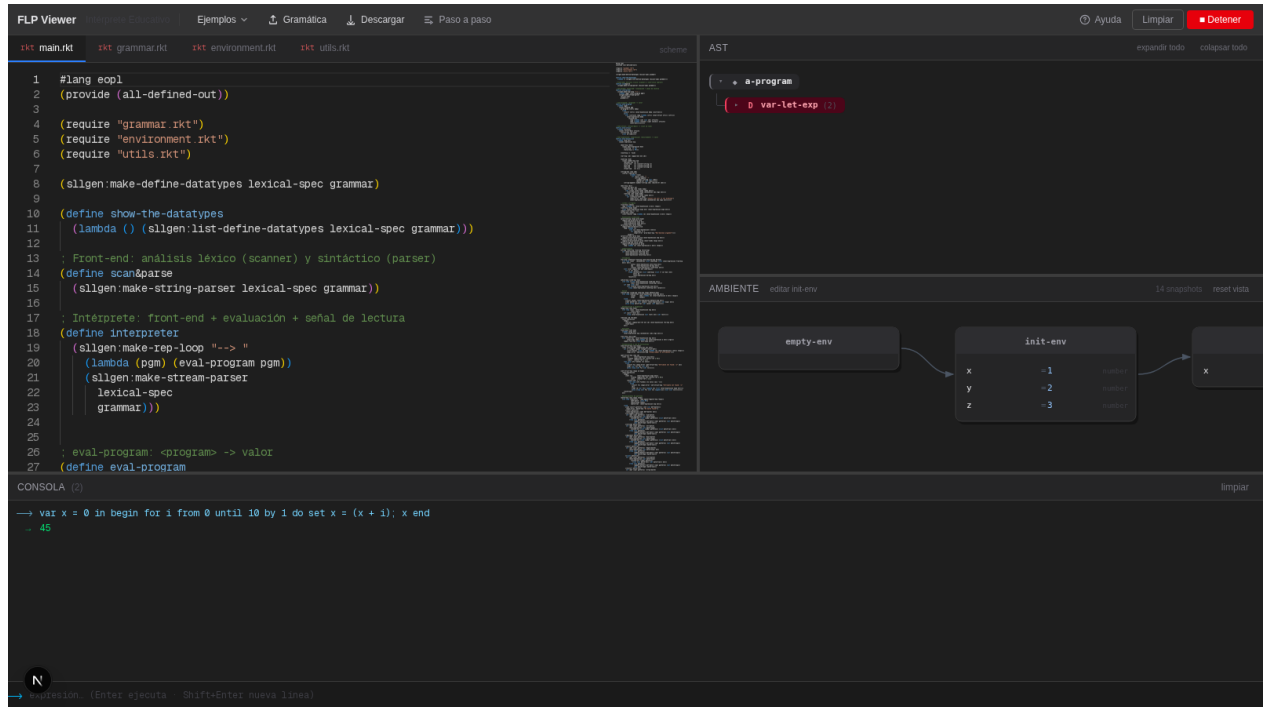


Figura 5.4: Evaluación de una expresión con iteración y estado mutable: suma acumulada mediante bucle `for`

Fuente: Elaboración propia

La Figura 5.5 evalúa la expresión `var f = func(x) if (x == 0) { 0 else (x + call f((x - 1))) } in call f(10)`, que define una función recursiva `f` y la aplica sobre 10, obteniendo la suma triangular 55. A diferencia del caso anterior, el marco final del ambiente contiene la ligadura `f` cuyo valor es una clausura, lo que permite al estudiante observar que en el modelo de EOPL las funciones son valores de primera clase almacenados en el ambiente como cualquier otra ligadura [4]. La presencia de la clausura en el panel de ambiente distingue esta visualización de la anterior y justifica su inclusión como caso complementario.

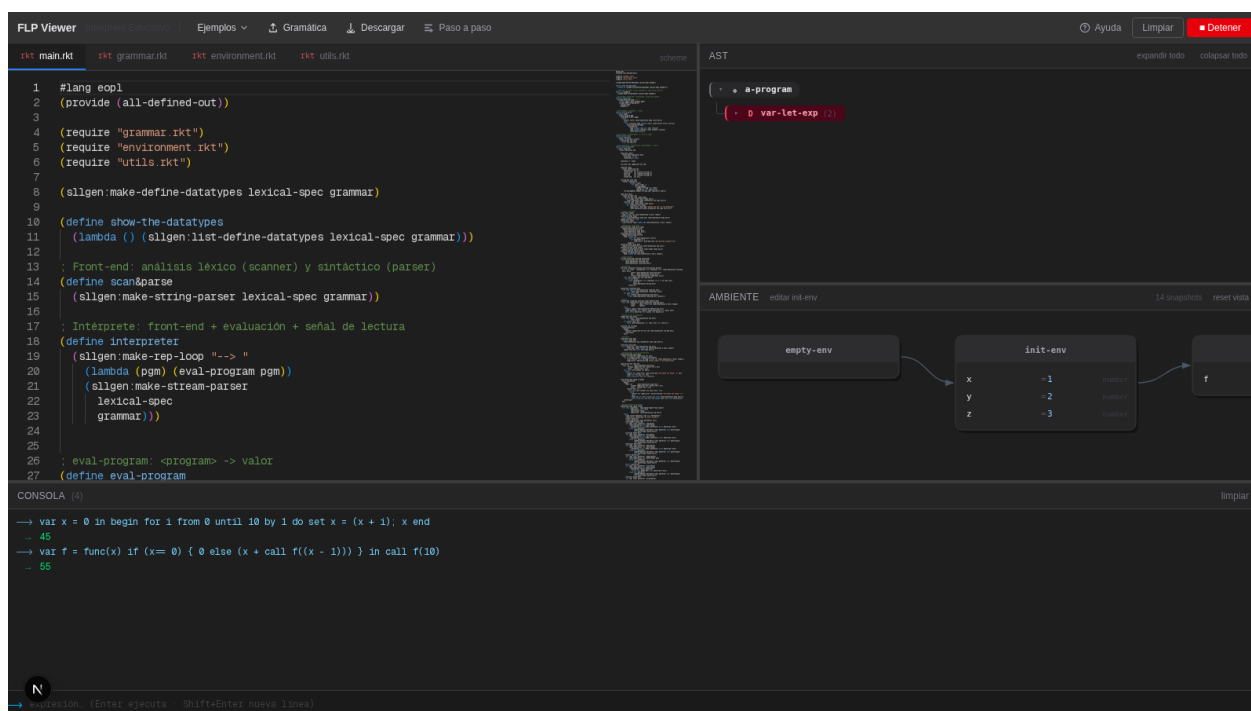


Figura 5.5: Evaluación de una función recursiva con clausura: la ligadura `f` en el panel de ambiente refleja una función como valor de primera clase

Fuente: Elaboración propia

5.2.3. Biblioteca de ejemplos predefinidos

La biblioteca de ejemplos predefinidos (RF-05) comprende seis programas cargados durante el renderizado inicial del servidor mediante `load-examples.ts` y presentados al estudiante desde el modal de bienvenida. Cada entrada está compuesta por el conjunto completo de archivos del intérprete (`grammar.rkt`, `environment.rkt`, `main.rkt` y `utils.rkt`), de modo que cualquier ejemplo puede ejecutarse y explorarse de forma inmediata sin modificaciones previas. La Tabla 5.1 presenta la correspondencia entre los ejemplos implementados y los indicadores de logro identificados en el Capítulo 3.

Tabla 5.1: Biblioteca de ejemplos predefinidos y correspondencia con indicadores de logro

#	Ejemplo	IL abordados
1	Hola Mundo	IL 1.3.1
2	Lenguaje LET	IL 1.1.6, IL 1.2.3
3	Ambientes	IL 1.2.6, IL 1.2.7
4	Procedimientos y cierres	IL 1.2.6
5	Estado y asignación	IL 1.2.6, IL 1.2.7
6	Intérprete EOPL (avanzado)	IL 1.1.6, IL 1.2.6, IL 1.2.7

Fuente: elaboración propia a partir del microcurrículo de la asignatura (código 750017C).

La secuencia de los ejemplos responde a dos ejes de variación simultáneos: la complejidad del lenguaje interpretado y la arquitectura del ambiente de ejecución. Los tres primeros ejemplos utilizan un

ambiente funcional cuyos valores se almacenan en listas inmutables. A partir del cuarto, el ambiente introduce celdas mutables mediante el tipo **a-ref** sobre vectores. Este cambio arquitectónico en **environment.rkt** es el que hace observable la diferencia entre ligadura y asignación, conceptos que en el modelo funcional no tienen representación estructural distinta [4]. Los párrafos siguientes describen el propósito específico de cada ejemplo dentro de esta progresión.

Hola Mundo. Con dos reglas gramaticales (**print-exp** y **string-exp**) y dos casos de evaluación, este ejemplo expone el pipeline completo de interpretación (gramática $\rightarrow$ AST $\rightarrow$ evaluación) en su forma más reducida, sin ligaduras activas ni operaciones sobre el ambiente. Su función en la biblioteca no es introducir un concepto nuevo sino permitir que el estudiante recorra cada etapa del proceso, desde el ingreso de la cadena de texto hasta la obtención del valor de retorno, sin que ningún elemento adicional interfiera en la lectura. Este diseño sigue el principio de gradación de la complejidad identificado en herramientas educativas para la enseñanza de compiladores [3], y garantiza que el primer contacto con el prototipo esté asociado a una experiencia de éxito inmediato, reduciendo así la barrera de entrada a la herramienta [7].

Lenguaje LET. Este ejemplo implementa el lenguaje LET descrito en el Capítulo 3 de EOPL [4], que constituye el primer intérprete del libro en el que el ambiente cumple una función activa. La gramática define siete reglas de producción y el evaluador implementa seis casos, incluyendo **let-exp**, que introduce la primera llamada a **extend-env**. El ambiente se implementa como una estructura funcional e inmutable: cada ligadura se almacena en un nuevo frame que se apila sobre el anterior sin modificar ninguno de los existentes. La relevancia de este ejemplo reside en que establece la correspondencia visible entre tres niveles: el nodo **let-exp** en el AST, la llamada a **extend-env** en el evaluador y la aparición del nuevo frame en el panel de ambiente, haciendo observable la mecánica básica del modelo de ambientes antes de introducir cualquier forma de estado mutable.

Ambientes. Este ejemplo comparte gramática y evaluador con el Lenguaje LET, pero su **init-env** precarga tres ligaduras (**x=1, y=2, z=3**). Esta diferencia, localizada exclusivamente en **environment.rkt**, hace que el panel de ambiente muestre de inmediato una cadena de frames activos, lo que permite observar dos comportamientos que en el Lenguaje LET requieren expresiones más elaboradas. Sorva et al. señalan que la efectividad de las visualizaciones de programas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [13]. La cadena de frames preexistente en este ejemplo ofrece ese modelo desde el primer contacto, sin que el estudiante deba construirlo mediante expresiones anidadas. El primero es la búsqueda de identificadores, que escala por la cadena desde el frame más reciente hasta el ambiente inicial. El segundo es el *shadowing*: al evaluar **let x = 10 in x**, el panel muestra un frame exterior con **x=1** y uno interior con **x=10**, haciendo visible que la ligadura interior no reemplaza a la exterior sino que la oculta dentro de su alcance [4].

Procedimientos y cierres. Este ejemplo extiende el Lenguaje LET con **proc** y **call**, correspondientes al lenguaje PROC del Capítulo 3 de EOPL [4]. La clausura se implementa como una función Racket que captura el ambiente léxico en el momento de la evaluación del **proc-exp**: el entorno activo queda embebido en el valor resultante y acompaña al procedimiento durante toda su vida. Este mecanismo hace visible un aspecto del modelo de EOPL que el código fuente por sí solo no comunica: en el panel de ambiente, el identificador ligado a la función no contiene un número

ni un booleano, sino un procedimiento, lo que evidencia que las funciones son valores de primera clase almacenados en el ambiente de la misma forma que cualquier otro dato [4]. Adicionalmente, al invocar la función, el evaluador extiende el ambiente capturado en la clausura y no el ambiente activo en el punto de llamada, lo que corresponde al modelo de alcance léxico descrito por Friedman y Wand [4] y permite que el estudiante verifique en el panel de ambiente qué ligaduras están disponibles durante la ejecución del cuerpo de la función.

Estado y asignación. Este ejemplo introduce la separación entre ligadura y asignación mediante el lenguaje IMPLICIT-REFS descrito en el Capítulo 4 de EOPL [4]. El cambio es arquitectónico y se concentra en `environment.rkt`: los valores dejan de almacenarse directamente en listas y pasan a residir en celdas mutables representadas por el tipo `a-ref` sobre vectores. Esta modificación habilita la operación `set`, implementada en `main.rkt` mediante `setref!`, que modifica el contenido de la celda sin crear un nuevo frame. La diferencia respecto a `let` se vuelve directamente verificable: evaluar `let x = 1 in let x = 2 in x` produce dos frames apilados en el panel de ambiente, mientras que evaluar `let x = 1 in begin set x = 2; x end` produce un único frame cuyo valor ha sido actualizado. Este contraste aborda directamente la dificultad específica reportada por el 77.8 % de los estudiantes encuestados con los conceptos de entorno y estado (P4, Tabla 3.1), y es coherente con la estrategia de hacer visibles los procesos internos del intérprete identificada como beneficiosa en la literatura sobre herramientas educativas para compiladores [3, 5].

Intérprete EOPL (avanzado). Este ejemplo presenta un intérprete de mayor escala que combina el modelo de referencias mutables con un lenguaje propio que incorpora estructuras de datos, arreglos, iteradores (`for`, `while`), `switch` y reconocimiento de patrones. A diferencia de los ejemplos anteriores, que aíslan conceptos específicos, este intérprete pone en juego simultáneamente los mecanismos de ligadura, clausura y estado mutable estudiados de forma separada en los ejemplos precedentes. La función `contains-set?`, incorporada en el evaluador, hace cumplir en tiempo de ejecución que las ligaduras `let` no admitan asignación, reforzando en el código ejecutable la distinción semántica respecto a `var` que el ejemplo anterior hizo observable a nivel del panel de ambiente. Este intérprete sirve además como base de las evaluaciones representativas documentadas en las Figuras 5.4 y 5.5.

5.2.4. Punto de acceso de ejecución

La ruta de API `/api/run` se implementó como un manejador HTTP POST en `app/api/run/route.ts`, siguiendo el modelo de rutas de Next.js. Su flujo de operación comprende cuatro fases.

En la primera fase, el servidor crea un directorio temporal exclusivo para la solicitud mediante `mkdtemp`⁴ y escribe en él todos los archivos del editor recibidos en el cuerpo de la petición. Antes de escribir `environment.rkt` y `main.rkt`, la función `injectRuntime` inyecta en su contenido los módulos de instrumentación correspondientes (`_tracking.rkt` y `_stream-parser.rkt` respectivamente). El archivo `_runner.rkt` se copia también al directorio temporal como punto de entrada del proceso Racket.

En la segunda fase, el servidor lanza el proceso Racket sobre `_runner.rkt` mediante `execFile`,

⁴`mkdtemp` es una llamada del sistema POSIX que crea un directorio temporal con nombre único, garantizando que no colisione con directorios preexistentes aunque varias solicitudes se procesen de forma concurrente.

pasando la expresión de prueba como argumento y aplicando un tiempo límite de 15 segundos. La señal de cancelación del `AbortController`⁵ asociado a la petición permite interrumpir el proceso si el cliente cancela la solicitud antes de que finalice la ejecución.

En la tercera fase, el servidor deserializa la salida estándar del proceso. Si la salida es un arreglo JSON válido, se interpreta como una secuencia de pasos de evaluación. De lo contrario, se trata como texto plano. Los mensajes de error presentes en `stderr` son normalizados por la función `cleanStderr`, que elimina las secciones de contexto interno del intérprete Racket que resultan irrelevantes para el estudiante.

En la cuarta fase, el servidor elimina el directorio temporal y retorna la respuesta HTTP con los campos `steps`, `stderr` y `error`. La eliminación del directorio se realiza en el bloque `finally` del manejador para garantizar la limpieza del sistema de archivos independientemente del resultado de la ejecución.

5.2.5. Instrumentación en tiempo de ejecución

La instrumentación en tiempo de ejecución se implementó mediante tres archivos Racket que se inyectan en el entorno de ejecución de forma transparente para el estudiante.

El archivo `_tracking.rkt` redefine la función `extend-env` al momento de cargar el módulo `environment.rkt`. Cada llamada a la función redefinida registra un *snapshot* del estado del ambiente en el log interno `_env-log`. Al finalizar cada evaluación, el ejecutor principal lee este log e incluye la secuencia de marcos en la respuesta serializada.

El archivo `_stream-parser.rkt` se inyecta al final de `main.rkt` e instrumenta la función de evaluación del programa para capturar, por cada reducción, el árbol de sintaxis abstracta resultante, el estado del ambiente y el valor producido. La salida se serializa como un arreglo JSON de objetos de paso, donde cada objeto incluye los campos `ast`, `environments` y `output`.

El archivo `_runner.rkt` actúa como punto de entrada del proceso Racket. Carga los módulos del proyecto en el orden correcto, recibe la expresión de prueba como argumento de línea de comandos y coordina la evaluación dentro de un manejador de excepciones que captura errores del intérprete y los incluye en la respuesta.

En el flujo de descarga del proyecto, la función `downloadZip` de `playground-utils.ts` genera un archivo ZIP con los archivos del editor, eliminando previamente los bloques de instrumentación marcados con las directivas `FLP-VIEWER-TRACKING-START/END`. Esto garantiza que el código exportado sea directamente compatible con el entorno DrRacket [12] sin modificaciones adicionales.

5.3. Flujo de ejecución principal

La Figura 5.6 ilustra el flujo de ejecución que se produce cuando el estudiante envía una expresión desde la consola del prototipo. El flujo inicia en el componente `ConsoleOutput`, que delega la acción al orquestador `PlaygroundLayout`. Este registra la expresión en el historial, inicializa el

⁵`AbortController` es una API web estándar que permite cancelar operaciones asíncronas mediante una señal compartida entre el código que inicia la operación y el que la ejecuta. Cuando el cliente cierra la conexión, la señal se activa y el proceso Racket es terminado antes de consumir el tiempo límite completo.

mecanismo de cancelación y deshabilita nuevas ejecuciones concurrentes. A continuación, invoca el servicio `racket.ts`, que empaqueta los archivos del editor y la expresión en una petición HTTP POST hacia `/api/run`. La solicitud transita por el contenedor **nginx**, que actúa como *reverse proxy* y la redirige al servidor de aplicación. El servidor prepara el entorno de ejecución aislado, lanza el proceso Racket, deserializa su salida y devuelve la secuencia de pasos al cliente. Finalmente, `PlaygroundLayout` distribuye los resultados entre los paneles de visualización según el modo activo: en modo normal actualiza el AST, el ambiente y la consola con el estado final, y en modo paso a paso encola los pasos y presenta el primero de forma inmediata, habilitando la navegación manual.

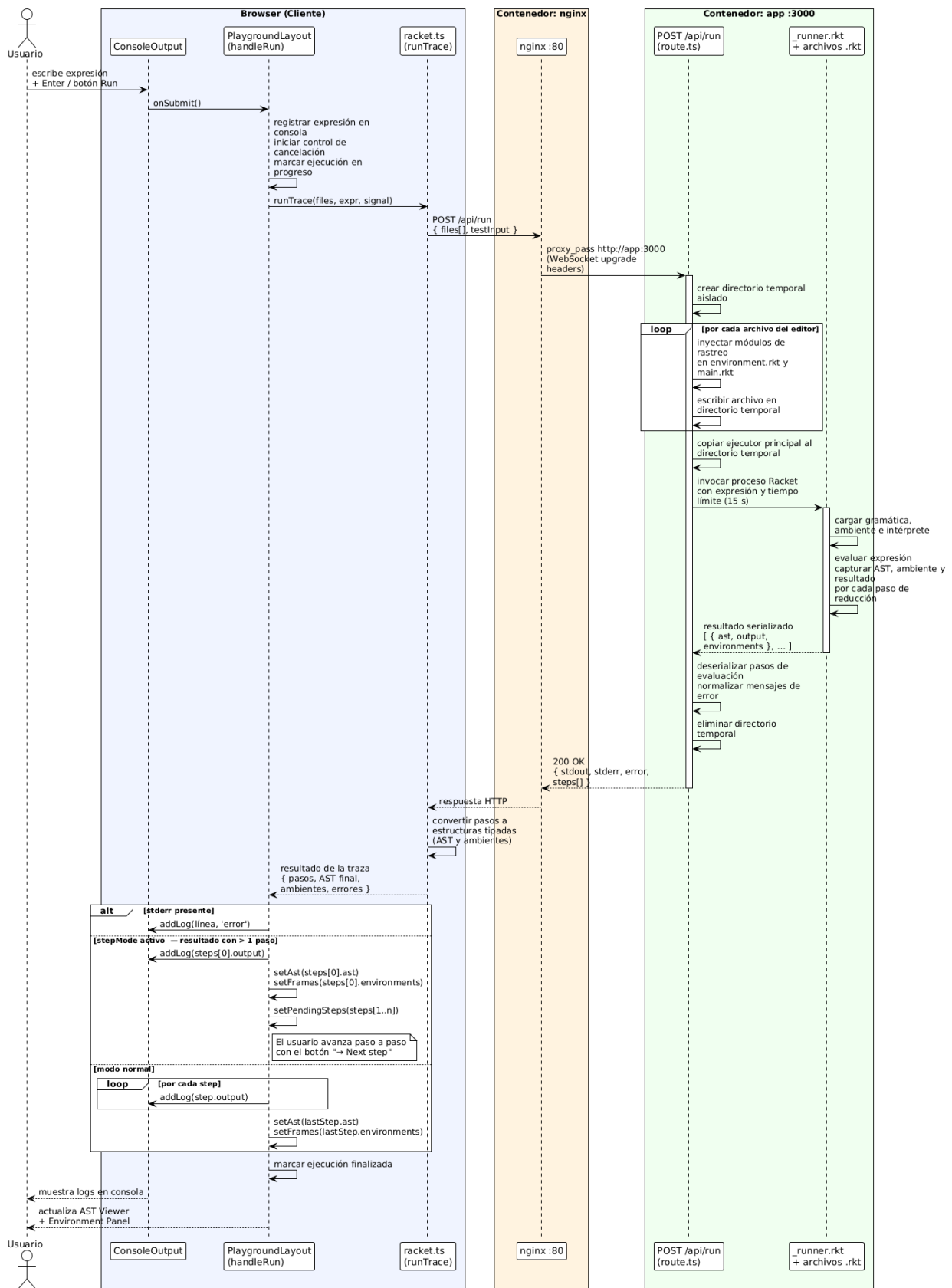


Figura 5.6: Diagrama de secuencia del flujo principal de ejecución del prototipo FLP Viewer

Fuente: Elaboración propia

5.4. Despliegue en entorno de pruebas

El prototipo se despliega mediante Docker Compose [32] en un único servidor anfitrión, con dos contenedores coordinados en la misma red interna. La Figura 5.7 presenta el diagrama de la infraestructura de despliegue.

La imagen del contenedor de aplicación se construye mediante un *Dockerfile* multi-etapa con tres etapas diferenciadas. La primera etapa (**deps**) instala las dependencias del proyecto utilizando **pnpm** sobre una imagen base de Node.js 25 Alpine. La segunda etapa (**builder**) ejecuta la construcción de Next.js en modo *standalone*⁶, que produce un artefacto autocontenido con todas las dependencias necesarias para ejecutar el servidor en producción. La tercera etapa (**runner**) utiliza la imagen **node:25-bookworm-slim** e instala el intérprete de Racket como dependencia del sistema operativo mediante **apt-get**. Esta etapa copia únicamente el artefacto *standalone*, el directorio de archivos estáticos, el directorio **public** y los tres archivos de instrumentación Racket, minimizando el tamaño de la imagen final. La variable de entorno **RACKET_BIN** se configura en **/usr/bin/racket** para que la ruta de API localice el ejecutable sin depender de la configuración del sistema anfitrión.

El servicio **app** expone el puerto interno 3000 al servicio **nginx** dentro de la red interna de Docker, pero no lo publica hacia el host. El servicio **nginx** utiliza la imagen oficial **nginx:alpine** y es el único que publica un puerto hacia el exterior (puerto 80). Su configuración se suministra como volumen de solo lectura mediante el archivo **nginx.conf**, que define una directiva **proxy_pass**⁷ hacia **http://app:3000** con los encabezados necesarios para el correcto funcionamiento del protocolo HTTP/1.1 y las conexiones *WebSocket*. Esta arquitectura garantiza que el acceso externo al prototipo se realice exclusivamente a través del proxy, centralizando el control de las solicitudes entrantes.

⁶El modo *standalone* de Next.js genera un servidor Node.js autocontenido que incluye únicamente las dependencias estrictamente necesarias para producción, sin requerir la carpeta **node_modules** completa en el entorno de despliegue. Esto reduce significativamente el tamaño de la imagen Docker resultante.

⁷**proxy_pass** es la directiva de nginx que especifica la dirección del servidor al que se redirigen las solicitudes entrantes. En combinación con los encabezados **Upgrade** y **Connection**, habilita además el soporte para conexiones *WebSocket*.

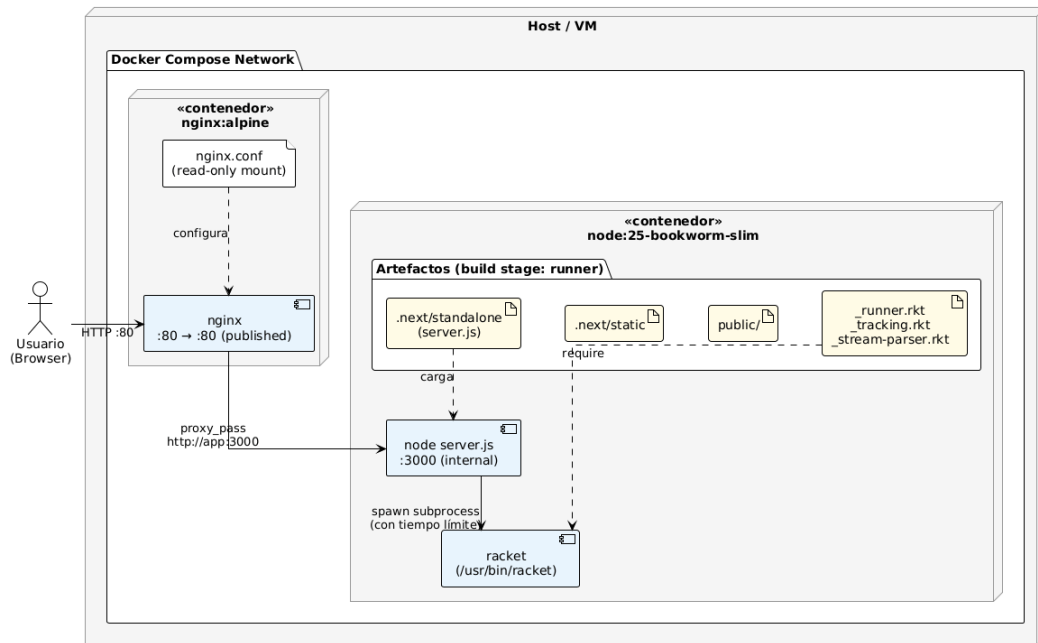


Figura 5.7: Diagrama de despliegue del prototipo FLP Viewer en el entorno de pruebas

Fuente: Elaboración propia

5.5. Verificación de requisitos funcionales

La Tabla 5.2 presenta la correspondencia entre los requisitos funcionales definidos en el diseño y los módulos que materializan cada uno en la implementación. Todos los requisitos de prioridad alta y media definidos en la especificación fueron implementados en el prototipo entregado.

Tabla 5.2: Verificación del cumplimiento de los requisitos funcionales

ID	Nombre	Módulos de implementación	Estado
RF-01	Generación del AST	<code>_runner.rkt</code> , <code>_stream-parser.rkt</code> , <code>racket.ts</code>	Implementado
RF-02	Visualización del AST	<code>ASTViewer.tsx</code> , <code>PlaygroundLayout.tsx</code>	Implementado
RF-03	Ejecución y evaluación	<code>app/api/run/route.ts</code> (<i>endpoint</i> <code>/api/run</code>)	Implementado
RF-04	Representación del ambiente	<code>_tracking.rkt</code> , <code>EnvironmentPanel.tsx</code>	Implementado
RF-05	Biblioteca de ejemplos	<code>examples/</code> , <code>load-examples.ts</code> , <code>WelcomeModal.tsx</code>	Implementado
RF-06	Validación de sintaxis Racket	<code>bnf-lexer.ts</code> , <code>bnf-parser.ts</code> , <code>cleanStderr()</code> en <code>route.ts</code>	Implementado
RF-07	Visualización del intérprete	<code>EditorPanel.tsx</code> , <code>CodeEditor.tsx</code> (líneas bloqueadas con Monaco Editor)	Implementado
RF-08	Plantilla base y exportación	<code>INITIAL_FILES</code> y <code>downloadZip()</code> en <code>playground-utils.ts</code>	Implementado

Fuente: elaboración propia.

Bibliografía

- [1] Q. Liu, W. Zhao, D. Liu, and J. Ji, “Evaluation of situational case-based teaching technique in engineering education,” in *Proceedings of the ACM Turing 50th Celebration Conference - China*, ACM TURC ’17, pp. 1–7, Association for Computing Machinery, May 2017.
- [2] E. A. Navas-López, “Modular and didactic compiler design with xml inter-phases communication,” *International Journal of Computer Science, Engineering and Information Technology (IJCEIT)*, vol. 12, pp. 1–15, February 2022.
- [3] S. Stamenković and N. Jovanović, “A web-based educational system for teaching compilers,” *IEEE Transactions on Learning Technologies*, vol. 17, pp. 143–156, 2024.
- [4] D. P. Friedman and M. Wand, *Essentials of Programming Languages, 3rd Edition*. The MIT Press, 3 ed., 2008.
- [5] L. Spigariol, J. Bono, and F. S. Guijarro, “Aprendiendo a desarrollar un intérprete de un lenguaje de programación funcional,” in *Actas del XXVIII Congreso Argentino de Ciencias de la Computación (CACIC)*, (La Rioja, Argentina), pp. 166–175, Universidad Tecnológica Nacional; Universidad Nacional de General San Martín; Universidad de Buenos Aires; Universidad Nacional de Hurlingham, Red de Universidades con Carreras en Informática, Oct. 2023.
- [6] N. Jovanović, S. Stamenković, and D. Miljković, “Comvis — interactive simulation environment for compiler learning,” *Computer Applications in Engineering Education*, vol. 30, no. 2, pp. 275–291, 2021.
- [7] J. Lu, M. Schmidt, M. Lee, and R. Huang, “Usability research in educational technology: A state-of-the-art systematic review,” *Educational Technology Research and Development*, vol. 70, no. 6, pp. 1951–1992, 2022.
- [8] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson Addison-Wesley, 2 ed., 2006.
- [9] J. Sweller, “Cognitive load theory and individual differences,” *Learning and Individual Differences*, vol. 110, p. 102423, 2024.
- [10] C. S. Gordon, “The linguistics of programming,” in *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! ’24)*, (New York, NY, USA), pp. 1–21, ACM, 2024.
- [11] J. Zagami, *Cognitive Influences on Learning Programming*, pp. 389–399. Springer, 02 2023.

- [12] M. Felleisen, R. B. Findler, M. Flatt, S. Krishnamurthi, E. Barzilay, J. McCarthy, and S. Tobin-Hochstadt, “A programmable programming language,” *Communications of the ACM*, vol. 61, no. 3, pp. 62–71, 2018.
- [13] J. Sorva, A. Karavirta, and L. Malmi, “A review of generic program visualization systems for introductory programming education,” *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013.
- [14] P. J. Guo, “Online Python Tutor: Embeddable web-based program visualization for CS education,” in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education*, pp. 579–584, ACM, 2013.
- [15] J. Clements, M. Flatt, and M. Felleisen, “Modeling an algebraic stepper,” in *Programming Languages and Systems (ESOP 2001)*, vol. 2028 of *Lecture Notes in Computer Science*, pp. 320–334, Springer, 2001.
- [16] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York: Basic Books, 2013.
- [17] C. Hannebauer, M. Hesenius, and V. Gruhn, “Does syntax highlighting help programming novices?,” *Empirical Software Engineering*, vol. 23, no. 4, pp. 1987–2013, 2018.
- [18] K. Cai, M. Henz, K.-L. Low, X. Y. Ng, J. R. Soh, K.-H. Tang, and K. W. Toh, “Visualizing environments of modern scripting languages,” in *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*, pp. 146–153, SCITEPRESS, 2023.
- [19] R. del Vado Vírveda, “Visualizing compiler design theory from implementation through an interactive tutoring tool: Experiences and results,” in *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*, pp. 333–340, SCITEPRESS, 2023.
- [20] K. M. Abad and M. Henz, “Beyond SICP — design and implementation of a notional machine for Scheme,” 2024.
- [21] B. Németh, E. Choi, E. Makihara, and H. Iida, “Investigating compilation errors of students learning haskell,” *Trends in Functional Programming in Education*, vol. 295, pp. 52–64, 2019.
- [22] “IEEE recommended practice for software requirements specifications,” Tech. Rep. IEEE Std 830-1998, Institute of Electrical and Electronics Engineers, New York, NY, USA, 1998.
- [23] “Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models,” Tech. Rep. ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.
- [24] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [25] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” *martinfowler.com*, March 2014. Consultado en 2025.
- [26] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*. Pearson, 4 ed., 2015.

- [27] Vercel, “Next.js — the react framework for the web.” <https://nextjs.org>, 2024. Versión 16.2.6.
- [28] Meta Open Source, “React — a javascript library for building user interfaces.” <https://react.dev>, 2024. Versión 19.
- [29] Microsoft, “Typescript — javascript with syntax for types.” <https://www.typescriptlang.org>, 2024. Versión 5.9.
- [30] Microsoft, “Monaco editor.” <https://microsoft.github.io/monaco-editor>, 2024. Versión 0.55.
- [31] Tailwind Labs, “Tailwind CSS — a utility-first CSS framework.” <https://tailwindcss.com>, 2024. Versión 4.
- [32] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, March 2014.

Anexo A

Microcurrículo de FLP

https://docs.google.com/spreadsheets/d/1loAXsbH8Y6nZ8b7QET8K_-sNE15NTxSg/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true

Anexo B

Encuesta a estudiantes

<https://docs.google.com/spreadsheets/d/1iAudDGoib0yQa7UZzohLvkJp29nifeoau/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true>

Anexo C

Encuesta a docentes

<https://docs.google.com/spreadsheets/d/1qeH1EeaJMBcqJZjNWuMYajdB9eqsvoa4XCK5dhJFenU/edit?gid=1616479297#gid=1616479297>

Anexo D

Protocolo de búsqueda sistemática de literatura

Este anexo documenta el protocolo de búsqueda sistemática utilizado para la identificación de los antecedentes presentados en el Capítulo 2, incluyendo las cadenas booleanas empleadas en cada base de datos académica, los criterios de inclusión y exclusión aplicados, el número de resultados obtenidos por base y las fechas de ejecución.

[Pendiente: completar con la bitácora de búsqueda una vez ejecutada la revisión sistemática.]

Anexo E

Requisitos del sistema

E.1. Requisitos Funcionales

Código	RF-01
Título	Generación del Árbol de Sintaxis Abstracta (AST)
Descripción	El sistema debe procesar el programa ingresado por el usuario utilizando la gramática previamente definida para generar su correspondiente Árbol de Sintaxis Abstracta (AST).
Prioridad	Alta
Criterios de aceptación	▪ El sistema invoca el intérprete Racket con los archivos del editor y la expresión de entrada; el AST resultante es serializado en formato JSON y transmitido al cliente. ▪ El AST generado es renderizado en el panel ASTViewer reflejando la estructura jerárquica del programa evaluado. ▪ Cuando el programa contiene errores sintácticos o de evaluación, el sistema muestra un mensaje de error en la consola con el prefijo × y nivel visual diferenciado en rojo; el panel ASTViewer conserva el último AST válido. ▪ El AST corresponde fielmente a la última expresión evaluada exitosamente.

Tabla E.1: RF-01: Generación del AST

Código	RF-02
Título	Visualización gráfica e interactiva del AST
Descripción	El sistema debe mostrar el árbol de sintaxis abstracta generado de forma gráfica e interactiva, permitiendo la exploración visual de su estructura.
Prioridad	Alta

Criterios de aceptación	■ El panel ASTViewer renderiza la representación jerárquica del AST con nodos que indican el tipo de dato retornado por el intérprete. ■ La visualización se actualiza automáticamente tras cada ejecución exitosa, sin necesidad de acción adicional por parte del usuario. ■ El panel es responsivo y se adapta al tamaño del contenedor asignado dentro del layout redimensionable. ■ En modo paso a paso, el panel refleja el AST correspondiente al paso de evaluación actualmente visualizado. ■ No se presentan errores de renderizado ni contenido huérfano durante el ciclo de vida de la sesión.
--------------------------------	--

Tabla E.2: RF-02: Visualización gráfica e interactiva del AST

Código	RF-03
Título	Ejecución y evaluación del programa
Descripción	El sistema debe permitir ejecutar el programa del usuario e interpretar su resultado, siguiendo el proceso de evaluación definido por la gramática y el intérprete generado.
Prioridad	Alta
Criterios de aceptación	■ El sistema transmite los archivos del editor y la expresión de entrada a la ruta <code>POST /api/run</code> y recibe el arreglo de pasos de evaluación. ■ El resultado de cada paso es presentado en la consola con nivel de salida correspondiente: verde (<code>output</code>) para valores, rojo (<code>error</code>) para excepciones capturadas, gris (<code>info</code>) para resultados <code>void</code>. ■ Los errores producidos por <code>eopl:error</code> son capturados por el runtime, limpiados de trazas internas de Racket, y presentados con el prefijo <code>*</code> en color rojo. ■ La ejecución puede ser cancelada por el usuario durante el proceso; el servidor aborta el subproceso Racket correspondiente. ■ El tiempo máximo de ejecución del subproceso Racket es de 15 segundos; transcurrido ese tiempo, el sistema reporta el error de tiempo límite en la consola. ■ El sistema soporta un modo de evaluación paso a paso que permite avanzar la ejecución de forma controlada mostrando el estado del AST y del ambiente en cada paso.

Tabla E.3: RF-03: Ejecución y evaluación del programa

Código	RF-04
Título	Representación del ambiente de ejecución
Descripción	El sistema debe representar el ambiente de evaluación como una estructura jerárquica de frames que se actualiza en cada ejecución, permitiendo al estudiante observar la evolución del estado del programa.
Prioridad	Alta
Criterios de aceptación	■ El panel EnvironmentPanel muestra el ambiente como una pila de frames jerárquicos, donde cada frame lista los bindings activos con su nombre, valor serializado y tipo de dato inferido. ■ El ambiente mostrado corresponde al último paso de evaluación completado; en modo paso a paso, se actualiza con cada avance del usuario. ■ La estructura de frames refleja fielmente la cadena de entornos construida durante la evaluación, permitiendo al estudiante identificar el alcance léxico de cada variable. ■ El panel permanece visible y no colapsa ante la ausencia de bindings en el ambiente actual.

Tabla E.4: RF-04: Representación del ambiente de ejecución

Código	RF-05
Título	Biblioteca de ejemplos
Descripción	El sistema debe ofrecer una biblioteca de ejemplos predefinidos orientados a los indicadores de logro identificados como base conceptual del prototipo, con el propósito de facilitar el aprendizaje mediante casos de uso concretos.
Prioridad	Media
Criterios de aceptación	■ Los ejemplos son cargados desde el sistema de archivos del servidor durante el renderizado inicial y transmitidos al cliente como propiedades del componente orquestador. ■ El sistema presenta la lista de ejemplos disponibles en el modal de bienvenida con nombre y descripción breve de cada uno. ■ Al seleccionar un ejemplo, los archivos del editor son reemplazados por los archivos correspondientes sin pérdida de estado de la interfaz. ■ Los ejemplos cubren los cinco indicadores de logro seleccionados en el diagnóstico: IL 1.1.6, IL 1.2.3, IL 1.2.6, IL 1.2.7 e IL 1.3.1. ■ Cada ejemplo es compatible con el pipeline de gramática y puede ser ejecutado sin modificaciones adicionales.

Tabla E.5: RF-05: Biblioteca de ejemplos

Código	RF-06
Título	Soporte y validación de sintaxis en Racket
Descripción	El sistema debe ser capaz de reconocer la sintaxis del lenguaje Racket en el editor y validar la gramática BNF ingresada por el usuario, reportando errores de forma clara y oportuna.
Prioridad	Alta
Criterios de aceptación	▪ El editor Monaco provee resaltado de sintaxis para archivos con extensión <code>.rkt</code>. ▪ El pipeline de gramática detecta y reporta errores léxicos en la especificación BNF durante el procesamiento; los mensajes son presentados en el modal de gramática antes de generar los archivos. ▪ El sistema no genera los archivos del intérprete ni habilita la ejecución cuando la gramática BNF contiene errores léxicos o sintácticos. ▪ Los errores de evaluación producidos por el intérprete Racket son presentados en la consola sin exponer trazas internas del runtime al usuario.

Tabla E.6: RF-06: Soporte y validación de sintaxis en Racket

Código	RF-07
Título	Visualización de los componentes internos del intérprete
Descripción	El sistema debe permitir al estudiante visualizar los archivos que componen el intérprete generado, mostrando su estructura con distinción entre las secciones técnicas y las secciones destinadas a la implementación del estudiante.
Prioridad	Media
Criterios de aceptación	▪ El panel de edición organiza los archivos del intérprete en pestañas independientes, accesibles mediante navegación por clic. ▪ Las líneas generadas automáticamente por el pipeline son marcadas como de solo lectura y presentadas con distinción visual respecto a las líneas editables. ▪ Las líneas destinadas a la implementación del estudiante (stubs de <code>eval-program</code>, <code>eval-expression</code>, <code>apply-primitive</code>) permanecen activas y editables. ▪ No se permite la modificación de las líneas protegidas durante la sesión de trabajo.

Tabla E.7: RF-07: Visualización de los componentes internos del intérprete

Código	RF-08
Título	Plantilla base editable y exportación del entorno
Descripción	El sistema debe proporcionar una plantilla base del intérprete donde el estudiante pueda completar la implementación, y permitir la descarga del proyecto completo en formato .zip compatible con la librería EOPL.
Prioridad	Alta
Criterios de aceptación	■ Al confirmar la gramática, el pipeline genera automáticamente los cuatro archivos del intérprete con los stubs de las funciones a implementar, cargándolos en el editor sin intervención adicional del usuario. ■ Las secciones de infraestructura técnica del intérprete están protegidas y no son modificables por el usuario. ■ Las secciones de implementación están identificadas con comentarios <code>;; TODO</code> que orientan al estudiante sobre qué debe completar. ■ El sistema genera un archivo .zip con todos los archivos del editor en su estado actual, incluyendo las modificaciones realizadas por el usuario. ■ El archivo descargado incluye una estructura de directorios compatible con la librería EOPL y puede ser abierto directamente en DrRacket.

Tabla E.8: RF-08: Plantilla base editable y exportación del entorno

E.2. Requisitos No Funcionales

Código	RNF-01
Título	Usabilidad
Descripción	El sistema debe ser intuitivo y operable por estudiantes sin experiencia avanzada en compiladores, minimizando la carga cognitiva extrínseca impuesta por la interfaz.
Prioridad	Alta

Criterios de aceptación	▪ Los usuarios pueden iniciar una sesión de trabajo, definir una gramática y ejecutar un programa sin consultar documentación externa. ▪ Los mensajes de error presentados en la consola son comprensibles sin conocimiento del funcionamiento interno de Racket. ▪ La distribución de paneles sigue un orden visual coherente con el flujo de trabajo del estudiante: definición de gramática, edición del intérprete, ejecución y observación de resultados. ▪ Las secciones del editor que no deben ser modificadas están diferenciadas visualmente de forma que el estudiante pueda identificarlas sin instrucción explícita.
--------------------------------	--

Tabla E.9: RNF-01: Usabilidad

Código	RNF-02
Título	Rendimiento
Descripción	El sistema debe completar la generación del AST y la evaluación del programa en un tiempo que no interrumpa el flujo cognitivo del estudiante durante la exploración.
Prioridad	Alta
Criterios de aceptación	▪ La transformación BNF a archivos Racket se completa en el cliente en menos de 1 segundo para gramáticas de hasta 20 producciones. ▪ La respuesta de la ruta <code>/api/run</code> para programas de complejidad habitual en la asignatura FLP se recibe en menos de 5 segundos bajo condiciones normales de red. ▪ El tiempo máximo de ejecución del subproceso Racket está limitado a 15 segundos; transcurrido ese tiempo, el sistema notifica al usuario y libera los recursos asociados. ▪ No se producen bloqueos del hilo principal del navegador durante la ejecución de la petición al servidor.

Tabla E.10: RNF-02: Rendimiento

Código	RNF-03
Título	Accesibilidad web
Descripción	El sistema debe ejecutarse en navegadores web modernos sin requerir instalación de software adicional, plugins ni autenticación por parte del usuario.
Prioridad	Alta

Criterios de aceptación	▪ La aplicación carga y opera correctamente en las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. ▪ No se requiere instalación de extensiones, plugins ni entornos de ejecución adicionales en el equipo del usuario. ▪ El acceso al sistema no requiere registro ni autenticación de ningún tipo. ▪ La interfaz es funcional en resoluciones de pantalla iguales o superiores a 1280×720 píxeles.
--------------------------------	---

Tabla E.11: RNF-03: Accesibilidad web

Código	RNF-04
Título	Mantenibilidad
Descripción	El código fuente debe estar organizado de forma modular, con separación explícita de responsabilidades, para facilitar la incorporación de mejoras futuras.
Prioridad	Media
Criterios de aceptación	▪ Las definiciones de tipos e interfaces TypeScript residen exclusivamente en el directorio <code>app/types/</code>, con importaciones explícitas desde los módulos que las consumen. ▪ Las funciones puras y utilitarios que no dependen del estado React residen en <code>app/lib/</code> y no en los archivos de componentes. ▪ El pipeline de gramática está segmentado en módulos independientes (lexer, parser, generadores) con responsabilidades no solapadas. ▪ Es posible añadir un nuevo generador de archivo Racket incorporando un módulo en <code>app/lib/</code> sin modificar los componentes de interfaz.

Tabla E.12: RNF-04: Mantenibilidad

Código	RNF-05
Título	Escalabilidad
Descripción	La arquitectura del sistema debe permitir incorporar nuevos ejemplos, gramáticas de mayor complejidad o características adicionales sin afectar la estabilidad de las funcionalidades existentes.
Prioridad	Media

Criterios de aceptación	■ La incorporación de nuevos ejemplos se realiza añadiendo archivos al directorio de ejemplos del servidor, sin modificar código de componentes ni de la API. ■ El pipeline de gramática procesa gramáticas con hasta 20 producciones sin degradación observable del rendimiento en el cliente. ■ La adición de un nuevo panel de visualización puede realizarse como un componente React independiente integrable en el layout sin alterar el comportamiento de los paneles existentes. ■ El sistema de contenedores Docker permite escalar el servicio de ejecución de forma independiente al servidor de la interfaz.
--------------------------------	--

Tabla E.13: RNF-05: Escalabilidad